pipeline/scoring.py

```
"""
scoring.py
----------
Responsibility:
- Accept a fully enriched meal dict (from pdf_extractor.py) and a patient's conditions
- Apply condition-specific multipliers grounded in NNR 2023 and Institutionskost 2025
- Return a letter grade A-E, a numeric score, clinical flags, and a per-nutrient breakdown

Nutrient thresholds (per meal, assuming 3 main meals/day at ~1800-2200 kcal/day):
Energy: 500-1000 kcal (target 700 kcal)
Protein: min 20 g (NNR 2023: 1.2 g/kg/day for elderly)
Fibre: min 5 g (NNR 2023: 25-35 g/day total)
Calcium: min 200 mg (NNR 2023: 950 mg/day for adults over 70)
Potassium: min 800 mg (NNR 2023: 3500 mg/day)
Sodium: max 750 mg (NNR 2023 CDRR: 2300 mg/day)
Saturated fat:max 12 g (NNR 2023: <10% of energy)
Sugar: max 25 g (NNR 2023: limit free sugars)

Note on sodium: all values are in mg throughout this module, matching Frida 5.5.

Usage:
from pipeline.scoring import score_meal, score_week

result = score_meal(enriched_meal, conditions=["Hypertension", "Osteoporosis"])
weekly = score_week(list_of_results)

Run directly to test:
python pipeline/scoring.py
"""
from typing import Dict, List
# ---------------------------------------------------------------------------
# Per-meal nutrient targets (NNR 2023 + Institutionskost 2025)
# min: below this is a deficiency penalty
# target: reaching this gives a full reward
# max: above this is an excess penalty
# ---------------------------------------------------------------------------
TARGETS = {
"energy_kcal": {"min": 500, "target": 700, "max": 1000, "unit": "kcal"},
"protein_g": {"min": 20, "target": 25, "max": None, "unit": "g"},
"fiber_g": {"min": 5, "target": 8, "max": None, "unit": "g"},
"calcium_mg": {"min": 200, "target": 300, "max": None, "unit": "mg"},
"sodium_mg": {"min": None, "target": 500, "max": 750, "unit": "mg"},
"fat_sat_g": {"min": None, "target": 8.0, "max": 12.0, "unit": "g"},
"sugar_g": {"min": None, "target": 12, "max": 25, "unit": "g"},
"potassium_mg": {"min": 800, "target": 1100, "max": None, "unit": "mg"},
}
# ---------------------------------------------------------------------------
# Condition multipliers
# Format: condition -> {nutrient: (direction, multiplier)}
# direction: "penalty" = bad nutrient gets penalised harder
# "reward" = good nutrient gets rewarded more
# multiplier: stacks multiplicatively across multiple conditions
# ---------------------------------------------------------------------------
CONDITION_MULTIPLIERS = {
"Hypertension": {
"sodium_mg": ("penalty", 2.5),
"fat_sat_g": ("penalty", 1.3),
"fiber_g": ("reward", 1.2),
},
"Type 2 Diabetes": {
"sugar_g": ("penalty", 2.5),
"fiber_g": ("reward", 1.5),
"energy_kcal": ("penalty", 1.3),
},
"Osteoporosis": {
"calcium_mg": ("reward", 2.0),
"protein_g": ("reward", 1.3),
},
"Malnutrition Risk": {
```

```
"protein_g": ("reward", 2.0),
"energy_kcal": ("reward", 0.5), # gentler energy penalty for low appetite
},
"High Cholesterol": {
"fat_sat_g": ("penalty", 2.0),
"fiber_g": ("reward", 1.5),
},
}
# ---------------------------------------------------------------------------
# Grade thresholds (lower score = better)
# ---------------------------------------------------------------------------
GRADE_THRESHOLDS = [
("A", float("-inf"), -2, "Excellent nutritional balance"),
("B", -2, 2, "Good — minor improvements possible"),
("C", 2, 7, "Adequate — improvements recommended"),
("D", 7, 13, "Poor — action recommended"),
("E", 13, float("inf"), "Critical — immediate review needed"),
]
# ---------------------------------------------------------------------------
# Internal helpers
# ---------------------------------------------------------------------------
def _get_multipliers(nutrient: str, conditions: List[str]):
"""Returns combined penalty and reward multipliers for a nutrient."""
penalty_mult = 1.0
reward_mult = 1.0
for cond in conditions:
effect = CONDITION_MULTIPLIERS.get(cond, {}).get(nutrient)
if effect:
direction, mult = effect
if direction == "penalty":
penalty_mult *= mult
else:
reward_mult *= mult
return penalty_mult, reward_mult
def _score_nutrient(nutrient: str, value: float, conditions: List[str]):
"""
Calculates the score contribution for one nutrient.
Returns:
contribution (float): negative = good (reward), positive = bad (penalty)
flags (list[str]): clinical alert messages for this nutrient
"""
t = TARGETS[nutrient]
penalty_mult, reward_mult = _get_multipliers(nutrient, conditions)
contribution = 0.0
flags = []
# --- Excess penalty (value above max) ---
if t["max"] is not None and value >= t["max"]:
overage = max(value - t["target"], 0)
base = (overage / t["max"]) * 5
contribution += min(base * penalty_mult, 8.0) # cap so no single nutrient dominates
if nutrient == "sodium_mg":
flags.append(
f"■ Sodium {value:.0f} mg exceeds limit ({t['max']} mg). "
"Review sodium-rich components."
)
elif nutrient == "fat_sat_g":
flags.append(
f"■ Saturated fat {value:.1f} g exceeds limit ({t['max']} g). "
"Consider lower-fat alternatives."
)
elif nutrient == "sugar_g":
flags.append(
f"■ Sugar {value:.1f} g exceeds limit ({t['max']} g). "
"Review dessert and sweetened components."
)
elif nutrient == "energy_kcal":
flags.append(
f"■ Energy {value:.0f} kcal exceeds recommended range ({t['max']} kcal). "
"Review portion sizes."
)
# --- Graduated penalty for target→max zone (penalty-only nutrients) ---
```

```
if t["min"] is None and t["max"] is not None and t["target"] < value < t["max"]:
zone_ratio = (value - t["target"]) / max(t["max"] - t["target"], 0.001)
partial = zone_ratio * 2.0 * penalty_mult
contribution += partial

# --- Deficiency penalty (value below min) ---
if t["min"] is not None and value < t["min"]:
shortfall = t["min"] - value
base = (shortfall / t["min"]) * 3
contribution += base * penalty_mult
if nutrient == "protein_g":
flags.append(
f"■ Protein {value:.1f} g below minimum ({t['min']} g). "
"Risk of muscle loss in elderly patients."
)
elif nutrient == "fiber_g":
flags.append(
f"■ Fibre {value:.1f} g below minimum ({t['min']} g). "
"Consider adding whole grains or vegetables."
)
elif nutrient == "calcium_mg":
flags.append(
f"■ Calcium {value:.0f} mg below minimum ({t['min']} mg). "
"Below recommended level for bone health."
)
elif nutrient == "energy_kcal":
flags.append(
f"■ Energy {value:.0f} kcal below minimum ({t['min']} kcal). "
"May indicate insufficient portion size."
)
# --- Reward (value meets or approaches target) ---
if t["min"] is not None and value >= t["min"]:
if value >= t["target"]:
base = 2.0
else:
ratio = (value - t["min"]) / max(t["target"] - t["min"], 0.001)
base = 1.0 * ratio
contribution -= base * reward_mult
return round(contribution, 3), flags
def _score_to_grade(score: float):
"""Maps a numeric score to a letter grade and description."""
for grade, low, high, desc in GRADE_THRESHOLDS:
if low <= score < high:
return grade, desc
return "E", "Critical — immediate review needed"
# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------
def score_meal(meal_or_nutrients, conditions: List[str]) -> Dict:
"""
Scores a meal for a patient with the given health conditions.
Args:
meal_or_nutrients:
Either a fully enriched meal dict (from pdf_extractor.extract_meals)
which has a "nutrients" key, or a plain nutrients dict directly.
conditions:
List of condition name strings, e.g. ["Hypertension", "Osteoporosis"].
Must match keys in CONDITION_MULTIPLIERS.
Returns dict with:
score (float): raw numeric score — lower is better
grade (str): letter grade A-E
grade_description (str): human-readable explanation
flags (list[str]): clinical alert messages
nutrient_scores (dict): per-nutrient score contributions
condition_effects (list): which conditions affected which nutrients
nutrients (dict): the input nutrient values
conditions (list): the input conditions
"""
# Accept either an enriched meal dict or a plain nutrients dict
if "nutrients" in meal_or_nutrients:
nutrients = meal_or_nutrients["nutrients"]
```

```
else:
nutrients = meal_or_nutrients
total_score = 0.0
all_flags = []
nutrient_scores = {}
for nutrient in TARGETS:
value = float(nutrients.get(nutrient, 0.0))
contribution, flags = _score_nutrient(nutrient, value, conditions)
total_score += contribution
nutrient_scores[nutrient] = contribution
all_flags.extend(flags)
grade, grade_description = _score_to_grade(total_score)
# Record which conditions affected which nutrients
condition_effects = []
for cond in conditions:
if cond in CONDITION_MULTIPLIERS:
condition_effects.append({
"condition": cond,
"affects": list(CONDITION_MULTIPLIERS[cond].keys()),
})
return {
"score": round(total_score, 2),
"grade": grade,
"grade_description": grade_description,
"flags": all_flags,
"nutrient_scores": nutrient_scores,
"condition_effects": condition_effects,
"nutrients": nutrients,
"conditions": conditions,
}
def score_week(meal_results: List[Dict]) -> Dict:
"""
Calculates a weekly summary from a list of score_meal() results.
Args:
meal_results: list of dicts as returned by score_meal()
Returns dict with:
weekly_avg_score (float)
weekly_grade (str)
weekly_description (str)
grade_counts (dict): {"A": n, "B": n, ...}
avg_nutrients (dict): average nutrient values across all meals
best_meal_index (int): index of the best-scored meal
worst_meal_index (int): index of the worst-scored meal
all_flags (list[str]): deduplicated flags across all meals
meal_count (int)
"""
if not meal_results:
return {}
scores = [r["score"] for r in meal_results]
avg_score = round(sum(scores) / len(scores), 2)
grade, desc = _score_to_grade(avg_score)
grade_counts = {"A": 0, "B": 0, "C": 0, "D": 0, "E": 0}
for r in meal_results:
grade_counts[r["grade"]] += 1
nutrient_keys = list(TARGETS.keys())
avg_nutrients = {}
for key in nutrient_keys:
values = [float(r["nutrients"].get(key, 0)) for r in meal_results]
avg_nutrients[key] = round(sum(values) / len(values), 2)
# Deduplicated flags
seen = set()
all_flags = []
for r in meal_results:
for flag in r["flags"]:
if flag not in seen:
seen.add(flag)
all_flags.append(flag)
return {
"weekly_avg_score": avg_score,
"weekly_grade": grade,
"weekly_description": desc,
```

```
"grade_counts": grade_counts,
"avg_nutrients": avg_nutrients,
"best_meal_index": scores.index(min(scores)),
"worst_meal_index": scores.index(max(scores)),
"all_flags": all_flags,
"meal_count": len(meal_results),
}
# ---------------------------------------------------------------------------
# Run directly to test
# ---------------------------------------------------------------------------
if __name__ == "__main__":
print("Testing scoring.py")
print("=" * 55)
# These nutrient keys match what nutrient_lookup.py returns
test_cases = [
{
"label": "Bøf stroganoff — Erik (Hypertension + Type 2 Diabetes)",
"nutrients": {
"energy_kcal": 680, "protein_g": 38, "sugar_g": 6,
"fiber_g": 4, "fat_sat_g": 9.0, "calcium_mg": 180,
"sodium_mg": 820, "potassium_mg": 600,
},
"conditions": ["Hypertension", "Type 2 Diabetes"],
},
{
"label": "Frikadeller — Birthe (Osteoporosis)",
"nutrients": {
"energy_kcal": 620, "protein_g": 42, "sugar_g": 3,
"fiber_g": 6, "fat_sat_g": 6.5, "calcium_mg": 210,
"sodium_mg": 540, "potassium_mg": 580,
},
"conditions": ["Osteoporosis"],
},
{
"label": "Fiskefrikadeller — Holger (no conditions)",
"nutrients": {
"energy_kcal": 520, "protein_g": 35, "sugar_g": 4,
"fiber_g": 5, "fat_sat_g": 4.0, "calcium_mg": 190,
"sodium_mg": 480, "potassium_mg": 500,
},
"conditions": [],
},
]
results = []
for tc in test_cases:
result = score_meal(tc["nutrients"], tc["conditions"])
results.append(result)
print(f"\n{tc['label']}")
print(f" Grade: {result['grade']} — {result['grade_description']}")
print(f" Score: {result['score']}")
if result["flags"]:
for flag in result["flags"]:
print(f" {flag}")
else:
print(" No flags.")
print("\n" + "=" * 55)
print("Weekly summary (all 3 meals)")
weekly = score_week(results)
print(f" Weekly grade: {weekly['weekly_grade']} — {weekly['weekly_description']}")
print(f" Avg score: {weekly['weekly_avg_score']}")
print(f" Grade counts: {weekly['grade_counts']}")
print(f" Best meal: index {weekly['best_meal_index']}")
print(f" Worst meal: index {weekly['worst_meal_index']}")
print(f" Unique flags: {len(weekly['all_flags'])}")
```

pipeline/frida_matcher.py

```
"""
frida_matcher.py
----------------
Responsibility:
- Translate informal Danish ingredient names from menu PDFs and the dish
catalogue into the exact food names used in the Frida 5.5 database
- Return a confidence score and flag low-quality matches for manual review

Matching strategy (three stages):
1. Manual override dictionary — known problem cases resolved immediately
at confidence 100, bypassing the algorithm entirely
2. Substring matching — find all Frida names containing the query as a
substring, then select the shortest (most generic) candidate
3. Fuzzy fallback — if no substring candidate exists, use rapidfuzz
token_set_ratio across the full Frida food list

Confidence threshold: 50. Matches below this are flagged for manual review
and stored in the ingredient_matches table with flagged_for_review=True.
The threshold is set at 50 rather than the conventional 80 because short
Danish ingredient names naturally produce lower raw similarity scores
against longer Frida entries, even when the match is correct.

Usage:
from pipeline.frida_matcher import load_frida, match_ingredient

frida_df = load_frida("data/Frida_5.5_Dataset.xlsx")
result = match_ingredient("kartoffel", frida_df)

Run directly to test:
python pipeline/frida_matcher.py
(prompts for path to Frida Excel file)
"""

# When prompted for Enter the path to the Frida Excel file: data/Frida_5.5_Dataset.xlsx

import pandas as pd
from rapidfuzz import process, fuzz

# If the match score is below this number, we flag it so someone can check it manually
# We use 50 instead of 80 because short Danish ingredient words like "2løg" or "2æg2" naturally score
lower against longer Frida names, even when the match is correct.
THRESHOLD = 50

# Manual overrides for ingredients that fuzzy matching consistently get wrong.
# Key: the ingredient name as it appears in dish_ingredients.json
# Value: the exact Danish name as it appears in the Frida Food sheet (FødevareNavn).
# Add new entries here whenever a flagged ingredient has a clearly wrong match.
MANUAL_OVERRIDES = {
"æg": "Æg, høne, æggehvide, rå",
"løg": "Løg, rå",
"oksekød": "Oksekød, inderlår uden kappe, rå",
"svinekød": "Kalv- og svinekød, hakket kalv og flæsk, 15-20% fedt, råt",
"sukker": "Sukker, stødt melis (sakkarose)",
"smør": "Smør, usaltet",
"fløde": "Fløde 38 %, piskefløde",
"mælk": "Sødmælk, konventionel (ikke-økologisk)",
"kartoffel": "Kartoffel, uspec., rå",
"ost": "Ost, fast, 45+, alle typer",
"salt": "Salt, bordsalt (jodberiget)",
"oksebouillon": "Bouillon, oksekød, terning, spiseklar",
"torsk": "Torsk, filet, rå",
"fiskefilet": "Torsk, filet, rå",
"ris": "Risengryn, grødris, brune, rå",
"pasta": "Pasta, rå",
"æbler": "Æble, uspec., råt",
"peberfrugt": "Peberfrugt, rød, rå",
"laks": "Laks, atlantisk, vild, rå",
"valnød": "Valnød, tørret",
"gulerod": "Gulerod, uspec., rå",
```

```
"tomat": "Tomat, uspec., rå",
"karry": "Karry, pulver",
"kaffe": "Kaffe, drikkeklar",
"yoghurt": "Yoghurt naturel, sødmælk",
"kalvekød": "Kalvekød, helt magert, råt",
"skinke": "Skinke, kogt, skiveskåret",
"kikærter": "Kikærter, tørrede, rå",
"kalkun": "Kalkun, kød, rå",
"bacon": "Bacon i skiver, rå",
"ymer": "Ymer naturel",
"ladyfinger": "Kiks, Marie",
"creme fraiche": "Creme fraiche 18 %",
"lammekød": "Lammekød, kølle, afpudset, rå",
"sennep": "Sennep, gul, færdiglavet",
"æble": "Æble, uspec., råt",
"cremefraiche": "Creme fraiche 18 %",
"kylling": "Kylling, kød, rå",
"gulerødder": "Gulerod, uspec., rå",
"surt": "Agurk, syltet",
"nannbrød": "Hvedebrød, pitabrød, fint",
"nannbrød med nigellafrø": "Hvedebrød, pitabrød, fint",
"stuvet ærter": "Grønne ærter, konserves",
"bailey's": "Likør, uspec.",
"hvid chokolade": "Chokolade, fløde",
"digestive": "Kiks, 20-30% fuldkorn, digestivetype",
"hønsekød": "Høne, kød, rå",


# plantefars, enebær, and vanilje have no accurate Frida 5.5 entry — will be flagged and left unmatched
}


# Load the food names and IDs from the Frida Excel file
def load_frida(filepath):
# Read the "Food" sheet from the Excel file
df = pd.read_excel(filepath, sheet_name="Food")

# Only keep the columns we need
df = df[["FødevareNavn", "FoodID"]]

# Remove rows where the food name is missing
df = df.dropna()

return df


# This function takes one ingredient name and finds the best match in Frida
def match_ingredient(ingredient, frida_df):
# Get all the Danish food names as a list
food_names = frida_df["FødevareNavn"].tolist()
query_lower = ingredient.strip().lower()

# Step 1: check the manual override dict first
# These are known good mappings for ingredients that fuzzy matching gets wrong because the query word
is short or appears inside many unrelated Frida names.
override = MANUAL_OVERRIDES.get(query_lower)
if override:
override_rows = frida_df[frida_df["FødevareNavn"] == override]
if not override_rows.empty:
frida_id = override_rows.iloc[0]["FoodID"]
return {
"query": ingredient,
"matched_name": override,
"frida_id": frida_id,
"confidence": 100.0,
"flagged_for_review": False
}


# Step 2: Find candidates whose Frida name contains the query as a substring.
# This filters out clearly irrelevant entries before scoring.
```

```
candidates = [
(i, name) for i, name in enumerate(food_names)
if query_lower in name.lower()
]

# Step 3: If no candidates contain the query as a substring (e.g. spelling variation
# or an ingredient name with no direct substring match), fall back to full fuzzy search.
if not candidates:
best_match, score, index = process.extractOne(ingredient, food_names, scorer=fuzz.token_set_ratio)
frida_id = frida_df.iloc[index]["FoodID"]
reported_score = round(score, 1)
return {
"query": ingredient,
"matched_name": best_match,
"frida_id": frida_id,
"confidence": reported_score,
"flagged_for_review": reported_score < THRESHOLD
}
# Step 4: among the substring candidates, pick the shortest name.
# The shortest matching Frida name is almost always the most generic raw ingredient.
# For example "Løg, rå" (7 chars) is preferred over "Løg, kogt, drænet" (18 chars).
best_index, best_name = min(candidates, key=lambda x: len(x[1]))

frida_id = frida_df.iloc[best_index]["FoodID"]
reported_score = round(fuzz.token_set_ratio(ingredient, best_name), 1)

result = {
"query": ingredient,
"matched_name": best_name,
"frida_id": frida_id,
"confidence": reported_score,
"flagged_for_review": reported_score < THRESHOLD
}

return result

# This function matches a whole list of ingredients at once
def match_all_ingredients(ingredients, frida_df):
all_results = []

# Loop through each ingredient and match it
for ingredient in ingredients:
result = match_ingredient(ingredient, frida_df)
all_results.append(result)

# Convert the list to a dataframe so it is easier to work with
results_df = pd.DataFrame(all_results)

return results_df


# Test the code with some example ingredients from the REH menu
if __name__ == "__main__":
# Ask the user for the path to the Frida file instead of hardcoding it
filepath = input("Enter the path to the Frida Excel file: ")

# Load the Frida database
frida_df = load_frida(filepath)

# Print how many foods were loaded
number_of_foods = len(frida_df)
print("Loaded " + str(number_of_foods) + " foods from Frida database")

# Some example ingredients from the dish_ingredients.json file
ingredients = [
"oksekød",
"champignon",
"creme fraiche",
"torskefilet",
"kartoffel",
"løg",
```

```
]

results = match_all_ingredients(ingredients, frida_df)

print("Match Results:")
print(results)

# Show which ingredients need to be checked manually
flagged = results[results["flagged_for_review"] == True]

if len(flagged) > 0:
# Print how many ingredients need to be checked
number_of_flagged = len(flagged)
print("Warning: " + str(number_of_flagged) + " ingredient(s) need manual review:")
print(flagged[["query", "matched_name", "confidence"]])
else:
print("All ingredients matched successfully!")

# data/Frida_5.5_Dataset.xlsx
```

pipeline/nutrient_lookup.py

```
"""
nutrient_lookup.py
------------------
Responsibility:
- Load the Frida 5.5 food composition dataset
- Retrieve nutrient values per 100 g using Frida ParameterIDs
- Scale nutrients by ingredient gram amounts
- Aggregate nutrients across multiple ingredients into a single meal profile

Design notes:
- Frida 5.5 stores nutrients as numeric ParameterIDs rather than stable
textual column names. Lookup uses these IDs directly for robustness.
- The dataset is cached in memory after the first load to avoid re-reading
the 12 MB Excel file on every pipeline call.

Usage:
from pipeline.nutrient_lookup import nutrients_for_food, aggregate_nutrients

n = nutrients_for_food("Torsk, filet, rå", grams=120)
total = aggregate_nutrients([n1, n2, n3])
"""

from pathlib import Path
from typing import Dict, List

import pandas as pd


# These values correspond to column headers when loading Data_Table with header=3.
NUTRIENT_COLUMNS = {
"energy_kcal": "356", # 356 → Energy (kcal)
"protein_g": "218", # 218 → Protein (g)
"fat_sat_g": "248", # 248 → Saturated fat (g)
"fiber_g": "168", # 168 → Fibre (g)
"sugar_g": "245", # 245 → Sugar (g)
"sodium_mg": "201", # 201 → Sodium (mg)
"potassium_mg": "165", # 165 → Potassium (mg)
"calcium_mg": "108", # 108 → Calcium (mg)
}


# --- INTERNAL DATASET CACHE ---
_FRIDA_DF = None


# --- DATASET LOADING ---
def load_frida_dataset() -> pd.DataFrame:
"""
Loads and normalizes the Frida 5.5 Data_Table.

Resulting DataFrame:
- index → Frida food names (e.g. 'Banana, rå')
- columns → Nutrient ParameterIDs (per 100 g)
"""
global _FRIDA_DF

if _FRIDA_DF is not None:
return _FRIDA_DF

project_root = Path(__file__).resolve().parents[1]
frida_path = project_root / "data" / "Frida_5.5_Dataset.xlsx"

if not frida_path.exists():
raise FileNotFoundError(f"Frida dataset not found: {frida_path}")

# Load Frida using the real nutrient header row and food-name column
df = pd.read_excel(
frida_path,
sheet_name="Data_Table",
```

```
header=3, # nutrient ParameterIDs
index_col=0, # food names
engine="openpyxl",
)

# Normalize food names
df.index = df.index.astype(str).str.strip()

# Normalize column names (ParameterIDs as strings)
df.columns = df.columns.astype(str).str.strip()

_FRIDA_DF = df
return df


# --- CORE NUTRIENT LOOKUP ---
def nutrients_for_food(food_name: str, grams: float) -> Dict[str, float]:
"""
Returns scaled nutrient values for a single ingredient,
identified by its canonical Frida food name.

Args:
food_name: Exact Frida food name (row index)
grams: Amount consumed (grams)

Returns:
Dictionary of nutrient values scaled to the given amount,
rounded to two decimals.
"""
df = load_frida_dataset()

if food_name not in df.index:
raise KeyError(f"No Frida entry found for food name: {food_name}")

row = df.loc[food_name]
scale = grams / 100.0

nutrients: Dict[str, float] = {}

for nutrient, parameter_id in NUTRIENT_COLUMNS.items():
if parameter_id not in df.columns:
raise KeyError(
f"Frida ParameterID not found in dataset: {parameter_id}"
)

value_per_100g = row[parameter_id]

scaled_value = (
0.0 if pd.isna(value_per_100g)
else float(value_per_100g) * scale
)

# Round all outputs to two decimals
nutrients[nutrient] = round(scaled_value, 2)

return nutrients


# --- AGGREGATION ---
def aggregate_nutrients(
nutrient_profiles: List[Dict[str, float]]
) -> Dict[str, float]:
"""
Aggregates multiple nutrient dictionaries by summation.

Returns:
Dictionary with summed nutrient totals rounded to two decimals.
"""
totals: Dict[str, float] = {}

for profile in nutrient_profiles:
```

```
for nutrient, value in profile.items():
totals[nutrient] = totals.get(nutrient, 0.0) + value

# Round aggregated totals
return {k: round(v, 2) for k, v in totals.items()}
```

pipeline/menu_parser.py

```
"""
menu_parser.py
--------------
Responsibility:
- Read a Madservice Roskilde menu PDF and extract each day's meal into a
structured dict that the rest of the pipeline can work with

The menu uses a two-column layout where the main dish and dessert appear on
the same line in separate columns. pdfplumber's word-position data is used
to split them by x-coordinate (threshold: 300).

Input: Path to a menu PDF (e.g. sample_data/menuplan-marts-2026.pdf)
Output: List of dicts, one per day, each with keys:
- day_number (int)
- day_name (str) e.g. "MAN"
- main (str) e.g. "Frikadeller"
- sides (list of str)
- dessert (str)
- vegetarian (bool)

Usage:
from pipeline.menu_parser import parse_menu

meals = parse_menu("sample_data/menuplan-marts-2026.pdf")

Run directly to test:
python pipeline/menu_parser.py sample_data/menuplan-marts-2026.pdf
"""

import re
import pdfplumber

# Words in the left column (main dish side) have x position below 300 and words in the right column
(dessert side) have x position above 300.
# We figured this out by printing the x positions of all words in the PDF.
COLUMN_SPLIT = 300

# The menu uses these abbreviations for the days of the week
DAY_NAMES = {"SØN", "MAN", "TIR", "ONS", "TOR", "FRE", "LØR"}
# These small words appear in the sides line but are not ingredients
WORDS_TO_IGNORE = {"med", "og", "samt", "på", "i", "af"}

def clean_sides(sides_text):
"""
Takes the raw sides text from the menu and returns a list of individual side dish names.
For example:
"Med kartoffelmos og drueagurker" becomes ["kartoffelmos", "drueagurker"]
"(Vegetarisk) med rodfrugter" becomes ["rodfrugter"]
"""
# Remove the vegetarian label if it is there
text = re.sub(r'\(vegetarisk\)', '', sides_text, flags=re.IGNORECASE)
text = text.strip()
# Remove leading connector words like "Med", "På" etc.
text = re.sub(r'^\s*(med|på|i|af)\s+', '', text, flags=re.IGNORECASE)
# Split the text on commas, "og", and "samt"
parts = re.split(r',|\bog\b|\bsamt\b', text, flags=re.IGNORECASE)
# Clean up each part and skip empty strings and connector words
result = []
for part in parts:
part = part.strip().lower()
if part and part not in WORDS_TO_IGNORE and len(part) > 2:
result.append(part)
return result

def is_vegetarian(text):
"""
Returns True if the text contains the vegetarian label.
"""
return bool(re.search(r'\(vegetarisk\)', text, re.IGNORECASE))
```

```
def get_rows_from_pdf(pdf_path):
"""
Opens the PDF and reads all the text rows.
Each row is returned as a tuple of (left_column_text, right_column_text).
We split by x position - words below 300 are in the left column (main dish), words above 300 are in the
right column (dessert).
Header and footer lines are skipped.
"""
all_rows = []
with pdfplumber.open(pdf_path) as pdf:
for page in pdf.pages:
words = page.extract_words()
if not words:
continue
# Group words into lines based on their vertical position. We round to the nearest 2 pixels because
words on the same line can have slightly different vertical positions in the PDF.
lines = {}
for word in words:
line_key = round(word['top'] / 2) * 2
if line_key not in lines:
lines[line_key] = []
lines[line_key].append(word)
# Go through each line from top to bottom
for line_key in sorted(lines.keys()):
line_words = lines[line_key]
# Split words into left and right column based on x position
left_words = []
right_words = []
for word in line_words:
if word['x0'] < COLUMN_SPLIT:
left_words.append(word['text'])
else:
right_words.append(word['text'])
left_text = ' '.join(left_words).strip()
right_text = ' '.join(right_words).strip()
# Skip empty lines
if not left_text:
continue
# Skip the header and footer lines
if left_text.startswith("MADSERVICE") or left_text.startswith("Periode"):
continue
if "allergener" in left_text.lower():
continue
all_rows.append((left_text, right_text))
return all_rows
def parse_menu(pdf_path):
"""
Main function. Takes a path to a menu PDF and returns a list of meal dictionaries, one for each day in
the menu.
"""
rows = get_rows_from_pdf(pdf_path)
meals = []
i = 0
while i < len(rows):
left_text, right_text = rows[i]
words = left_text.split()
# Check if this row starts with a day name like "MAN" or "TIR"
if words and words[0] in DAY_NAMES:
day_name = words[0]
# The main dish name is the rest of the left column
main = ' '.join(words[1:]).strip()
# The dessert is in the right column
dessert = right_text.strip()
# The next row should have the day number and the sides
day_number = None
sides = []
vegetarian = False
if i + 1 < len(rows):
next_left, next_right = rows[i + 1]
next_words = next_left.split()
# Check if the next row starts with a number (the day number)
```

```
if next_words and next_words[0].isdigit():
day_number = int(next_words[0])
# The sides are the rest of the text on that row
sides_text = ' '.join(next_words[1:])
vegetarian = is_vegetarian(sides_text)
sides = clean_sides(sides_text)
# Skip the sides row since we already read it
i += 1
meals.append({
"day_number": day_number,
"day_name": day_name,
"main": main,
"sides": sides,
"dessert": dessert,
"vegetarian": vegetarian
})
i += 1
return meals
# This block only runs when you execute this file directly. It's used for testing the module during
development.
# Example: python pipeline/menu_parser.py sample_data/menuplan-marts-2026.pdf
if __name__ == "__main__":
import sys
# Use the file path from the command line, or default to March menu
if len(sys.argv) > 1:
pdf_path = sys.argv[1]
else:
pdf_path = "sample_data/menuplan-marts-2026.pdf"
print("Parsing:", pdf_path)
print()
meals = parse_menu(pdf_path)
print("Found", len(meals), "meals")
print()
warnings = 0
for meal in meals:
veg_label = " [VEGETARISK]" if meal["vegetarian"] else ""
print("Day", meal["day_number"], meal["day_name"])
print(" Main: ", meal["main"])
print(" Dessert: ", meal["dessert"])
print(" Sides: ", meal["sides"])
print(" " + veg_label if veg_label else "")
# Basic checks
if not meal["main"]:
print(" WARNING: no main dish found")
warnings += 1
if not meal["dessert"]:
print(" WARNING: no dessert found")
warnings += 1
print()
print("Done.", len(meals), "meals parsed,", warnings, "warnings.")
```

pipeline/pdf_extractor.py

```
"""
pdf_extractor.py
----------------
Responsibility:
- Parse a Madservice Roskilde menu PDF into structured meal dicts (via menu_parser)
- Enrich each meal with ingredient data from dish_ingredients.json
- Match ingredient names to Frida via frida_matcher.py (fuzzy matching)
- Look up nutrient values for each enriched meal via nutrient_lookup.py
- Return fully enriched meal dicts ready for scoring.py
Pipeline flow:
PDF → menu_parser → dish_ingredients.json → frida_matcher → nutrient_lookup → scoring
Input: Path to a menu PDF (e.g. sample_data/menuplan-marts-2026.pdf)
Output: List of fully enriched meal dicts, one per day, with keys:
- day_number, day_name, main, sides, dessert, vegetarian
- main_ingredients, main_amounts_g
- dessert_ingredients, dessert_amounts_g
- nutrients (dict) aggregated from nutrient_lookup.py
- matched (list) ingredients matched in Frida
- unmatched (list) ingredients not matched
- flagged (list) low-confidence matches for review
- coverage_pct (int) % of ingredients matched
Usage:
from pipeline.pdf_extractor import extract_meals
meals = extract_meals("sample_data/menuplan-marts-2026.pdf")
Run directly to test:
python -m pipeline.pdf_extractor sample_data/menuplan-marts-2026.pdf
"""
import json
import sys
from pathlib import Path
try:
from pipeline.menu_parser import parse_menu
from pipeline.nutrient_lookup import nutrients_for_food, aggregate_nutrients
from pipeline.frida_matcher import load_frida, match_ingredient
except ImportError:
from menu_parser import parse_menu
from nutrient_lookup import nutrients_for_food, aggregate_nutrients
from frida_matcher import load_frida, match_ingredient
# ---------------------------------------------------------------------------
# Paths
# ---------------------------------------------------------------------------
PROJECT_ROOT = Path(__file__).resolve().parents[1]
DISH_INGREDIENTS_PATH = PROJECT_ROOT / "sample_data" / "dish_ingredients.json"
FRIDA_PATH = PROJECT_ROOT / "data" / "Frida_5.5_Dataset.xlsx"
# ---------------------------------------------------------------------------
# Caches
# ---------------------------------------------------------------------------
_DISH_DB = None
_FRIDA_DF = None
def _load_dish_db():
global _DISH_DB
if _DISH_DB is not None:
return _DISH_DB
if not DISH_INGREDIENTS_PATH.exists():
raise FileNotFoundError(
f"dish_ingredients.json not found at: {DISH_INGREDIENTS_PATH}"
)
with open(DISH_INGREDIENTS_PATH, encoding="utf-8") as f:
raw = json.load(f)
merged = {}
for category, dishes in raw.items():
if category == "_metadata":
continue
if isinstance(dishes, dict):
for dish_name, dish_data in dishes.items():
merged[dish_name] = dish_data
_DISH_DB = merged
return merged
def _load_frida_df():
```

```
"""Loads the Frida Food sheet for fuzzy matching (cached)."""
global _FRIDA_DF
if _FRIDA_DF is not None:
return _FRIDA_DF
if not FRIDA_PATH.exists():
raise FileNotFoundError(
f"Frida dataset not found at: {FRIDA_PATH}"
)
_FRIDA_DF = load_frida(str(FRIDA_PATH))
return _FRIDA_DF
# ---------------------------------------------------------------------------
# Dish matching
# ---------------------------------------------------------------------------
def _find_dish(dish_name, dish_db):
"""Finds ingredient data for a dish from dish_ingredients.json."""
if not dish_name:
return None
if dish_name in dish_db:
return dish_db[dish_name]
lower = dish_name.strip().lower()
for key, val in dish_db.items():
if key.lower() == lower:
return val
for key, val in dish_db.items():
if key.lower() in lower or lower in key.lower():
return val
return None
# ---------------------------------------------------------------------------
# Nutrient lookup via frida_matcher
# ---------------------------------------------------------------------------
EMPTY_NUTRIENTS = {
"energy_kcal": 0.0,
"protein_g": 0.0,
"fat_sat_g": 0.0,
"fiber_g": 0.0,
"sugar_g": 0.0,
"sodium_mg": 0.0,
"potassium_mg": 0.0,
"calcium_mg": 0.0,
}
def _lookup_component(name, ingredients, amounts_g):
"""
Matches each ingredient via frida_matcher, then looks up nutrients.
Returns:
nutrients (dict): aggregated nutrient totals
matched (list): successfully looked up ingredients
unmatched (list): ingredients that failed nutrient lookup
flagged (list): low-confidence fuzzy matches needing review
coverage_pct (int): % of ingredients with nutrient data
"""
frida_df = _load_frida_df()
profiles = []
matched = []
unmatched = []
flagged = []
for ingredient, amount in zip(ingredients, amounts_g):
# Step 1: fuzzy match ingredient name to a Frida food name
match_result = match_ingredient(ingredient, frida_df)
frida_name = match_result["matched_name"]
confidence = match_result["confidence"]
# Track low-confidence matches for reporting
if match_result["flagged_for_review"]:
flagged.append({
"ingredient": ingredient,
"matched_to": frida_name,
"confidence": confidence,
})
# Step 2: look up nutrients using the matched Frida name
try:
profile = nutrients_for_food(frida_name, amount)
profiles.append(profile)
```

```
matched.append({
"ingredient": ingredient,
"frida_name": frida_name,
"confidence": confidence,
"amount_g": amount,
})
except KeyError:
unmatched.append(ingredient)
nutrients = aggregate_nutrients(profiles) if profiles else dict(EMPTY_NUTRIENTS)
total = len(matched) + len(unmatched)
coverage = round(len(matched) / total * 100) if total > 0 else 0
return {
"nutrients": nutrients,
"matched": matched,
"unmatched": unmatched,
"flagged": flagged,
"coverage_pct": coverage,
}
# ---------------------------------------------------------------------------
# Meal enrichment
# ---------------------------------------------------------------------------
def _enrich_meal(meal, dish_db):
"""Enriches a raw meal dict with ingredients and nutrients."""
enriched = dict(meal)
all_profiles = []
all_matched = []
all_unmatched = []
all_flagged = []
# --- Main dish ---
main_data = _find_dish(meal.get("main", ""), dish_db)
if main_data:
main_ingredients = main_data.get("ingredients", [])
main_amounts = main_data.get("amounts_g", [100] * len(main_ingredients))
result = _lookup_component(meal["main"], main_ingredients, main_amounts)
all_profiles.append(result["nutrients"])
all_matched.extend(result["matched"])
all_unmatched.extend(result["unmatched"])
all_flagged.extend(result["flagged"])
enriched["main_ingredients"] = main_ingredients
enriched["main_amounts_g"] = main_amounts
enriched["main_coverage_pct"] = result["coverage_pct"]
else:
enriched["main_ingredients"] = []
enriched["main_amounts_g"] = []
enriched["main_coverage_pct"] = 0
# --- Sides (each side is a single ingredient, default 100g) ---
sides = meal.get("sides", [])
if sides:
# Specific portion overrides for sides where 100g is unrealistic.
# Condiments, bread, and sauces are served in much smaller amounts.
SIDE_PORTIONS = {
"rugbrød": 60,
"sennep": 15,
"ketchup": 20,
"citron": 20,
"cremefraiche": 30,
"rødkål": 60,
"asier": 40,
"drueagurker": 40,
"syltetøj": 20,
"ribsgele": 20,
"tyttebær": 30,
"agurkesalat": 60,
"waldorfsalat": 80,
"rødkålssalat": 60,
"hvidkålssalat": 60,
"gulerodssalat": 60,
"gulerodsråkost":60,
"råkost": 60,
"dild": 5,
"persille": 5,
```

```
"surt": 40,
"nannbrød": 60,
"nannbrød med nigellafrø": 60,
"stuvet ærter": 80,
"gulerødder": 80,
"stuvet bønner": 80,
"rødbeder": 60,
"champignon": 60,
}
side_amounts = [SIDE_PORTIONS.get(s.strip().lower(), 80) for s in sides]
result = _lookup_component("sides", sides, side_amounts)
all_profiles.append(result["nutrients"])
all_matched.extend(result["matched"])
all_unmatched.extend(result["unmatched"])
all_flagged.extend(result["flagged"])
# --- Dessert ---
dessert_data = _find_dish(meal.get("dessert", ""), dish_db)
if dessert_data:
dessert_ingredients = dessert_data.get("ingredients", [])
dessert_amounts = dessert_data.get("amounts_g", [100] * len(dessert_ingredients))
result = _lookup_component(meal["dessert"], dessert_ingredients, dessert_amounts)
all_profiles.append(result["nutrients"])
all_matched.extend(result["matched"])
all_unmatched.extend(result["unmatched"])
all_flagged.extend(result["flagged"])
enriched["dessert_ingredients"] = dessert_ingredients
enriched["dessert_amounts_g"] = dessert_amounts
enriched["dessert_coverage_pct"] = result["coverage_pct"]
else:
enriched["dessert_ingredients"] = []
enriched["dessert_amounts_g"] = []
enriched["dessert_coverage_pct"] = 0
# --- Aggregate full-meal nutrients ---
enriched["nutrients"] = aggregate_nutrients(all_profiles) if all_profiles else dict(EMPTY_NUTRIENTS)
enriched["matched"] = all_matched
enriched["unmatched"] = all_unmatched
enriched["flagged"] = all_flagged
total = len(all_matched) + len(all_unmatched)
enriched["coverage_pct"] = round(len(all_matched) / total * 100) if total > 0 else 0
return enriched
# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------
def extract_meals(pdf_path):
"""
Parses a menu PDF and returns fully enriched meal dicts ready for scoring.
Args:
pdf_path: Path to the Roskilde menu PDF
Returns:
List of enriched meal dicts, one per day.
"""
raw_meals = parse_menu(pdf_path)
dish_db = _load_dish_db()
return [_enrich_meal(meal, dish_db) for meal in raw_meals]
# ---------------------------------------------------------------------------
# Run directly to test
# python -m pipeline.pdf_extractor sample_data/menuplan-marts-2026.pdf
# ---------------------------------------------------------------------------
if __name__ == "__main__":
pdf_path = sys.argv[1] if len(sys.argv) > 1 else str(
PROJECT_ROOT / "sample_data" / "menuplan-marts-2026.pdf"
)
print(f"Extracting: {pdf_path}\n")
meals = extract_meals(pdf_path)
print(f"Found {len(meals)} meals\n")
warnings = 0
for meal in meals:
veg = " [VEGETARISK]" if meal.get("vegetarian") else ""
print(f"Day {meal['day_number']} {meal['day_name']}{veg}")
print(f" Main: {meal['main']} ({len(meal['main_ingredients'])} ingredients)")
print(f" Dessert: {meal['dessert']} ({len(meal['dessert_ingredients'])} ingredients)")
```

```
print(f" Sides: {meal['sides']}")
print(f" Coverage: {meal['coverage_pct']}%")
n = meal["nutrients"]
print(f" Nutrients: energy={n.get('energy_kcal',0):.0f} kcal "
f"protein={n.get('protein_g',0):.1f}g "
f"fibre={n.get('fiber_g',0):.1f}g "
f"calcium={n.get('calcium_mg',0):.0f}mg "
f"sodium={n.get('sodium_mg',0):.0f}mg")
if meal["flagged"]:
print(f" ■ Low-confidence matches:")
for f in meal["flagged"]:
print(f" {f['ingredient']} → {f['frida_name']} ({f['confidence']}%)")
if meal["unmatched"]:
print(f" ✗ Unmatched: {meal['unmatched']}")
warnings += 1
print()
print(f"Done. {len(meals)} meals, {warnings} with unmatched ingredients.")
```

pipeline/build_db.py

```
"""
build_db.py
-----------
Responsibility:
- Build the SQLite database (nutriscan.db) used by the NutriScan API and dashboard
- Orchestrates the pipeline modules to pre-compute and cache all matches and scores

The database contains six tables:
patients Synthetic resident profiles
patient_conditions One row per condition with status (primary / risk / inactive)
dishes One row per dish from dish_ingredients.json
dish_ingredients Ingredients and gram amounts for each dish
ingredient_matches Frida matcher output for every unique ingredient
meal_scores Grade and score from scoring.py for every patient-dish pair

A seventh table (score_history) is created at API startup by api/main.py
and populated at runtime when scoring events occur.

Run this file to (re)build the database from scratch:
python pipeline/build_db.py

Note: re-running deletes and recreates nutriscan.db entirely, so any
score_history entries written at runtime will be lost.
"""
import json
import sqlite3
import sys
from pathlib import Path
# Paths to the input files and where the database should be saved
# build_db.py lives inside pipeline/, so we go one level up to reach the project root
PROJECT_ROOT = Path(__file__).resolve().parents[1]
FRIDA_FILE = PROJECT_ROOT / "data" / "Frida_5.5_Dataset.xlsx"
DISH_FILE = PROJECT_ROOT / "sample_data" / "dish_ingredients.json"
DB_FILE = PROJECT_ROOT / "pipeline" / "nutriscan.db"
# Make sure we can import from the pipeline folder
sys.path.insert(0, str(PROJECT_ROOT))
from pipeline.scoring import score_meal
from pipeline.frida_matcher import load_frida, match_ingredient
# We import load_frida and match_ingredient from pipeline/frida_matcher.py rather
# than copying the logic here. That way the database always uses the same matching
# strategy as the rest of the system, including the manual overrides for ingredients
# like "aeg", "loeg", "oksekoed" etc. that WRatio consistently got wrong.
# Real nutrient lookup — uses the Frida Data_Table instead of synthetic baselines
from pipeline.nutrient_lookup import load_frida_dataset, nutrients_for_food, aggregate_nutrients
# The fake patients used by the dashboard, copied over so the database is self-contained.
# Each patient has an id, basic demographics, and a list of health conditions.
# Conditions are split into primary (active), risk (at risk), and inactive.
PATIENTS = [
{
"id": "XY-5217",
"name": "Erik Andersen",
"sex": "Male",
"age": 82,
"location": "Wing A, Room 3",
"avatar": "EA",
"conditions": {
"primary": ["Hypertension", "Type 2 Diabetes"],
"risk": ["Malnutrition Risk"],
"inactive": []
}
},
{
"id": "XY-3891",
"name": "Birthe Sorensen",
"sex": "Female",
"age": 79,
"location": "Wing B, Room 7",
"avatar": "BS",
"conditions": {
```

```
"primary": ["Osteoporosis"],
"risk": ["Malnutrition Risk"],
"inactive": ["Dementia"]
}
},
{
"id": "XY-4422",
"name": "Kaj Nielsen",
"sex": "Male",
"age": 85,
"location": "Wing C, Room 12",
"avatar": "KN",
"conditions": {
"primary": ["High Cholesterol"],
"risk": ["Hypertension"],
"inactive": []
}
},
{
"id": "XY-6104",
"name": "Grethe Larsen",
"sex": "Female",
"age": 76,
"location": "Wing A, Room 9",
"avatar": "GL",
"conditions": {
"primary": ["Malnutrition Risk"],
"risk": [],
"inactive": ["Type 2 Diabetes"]
}
},
{
"id": "XY-7733",
"name": "Holger Christensen",
"sex": "Male",
"age": 88,
"location": "Wing D, Room 2",
"avatar": "HC",
"conditions": {
"primary": [],
"risk": [],
"inactive": []
}
}
]
# Only pass conditions that scoring.py has multipliers for.
# Anything else (e.g. Dementia) is dropped before scoring.
KNOWN_CONDITIONS = {
"Hypertension",
"Type 2 Diabetes",
"Osteoporosis",
"Malnutrition Risk",
"High Cholesterol",
}
# Load the dishes from dish_ingredients.json and flatten them into simple lists
def load_dishes(filepath):
raw = json.loads(Path(filepath).read_text(encoding="utf-8"))
dishes = [] # (dish_name, category, kostform)
dish_ings = [] # (dish_name, ingredient, grams) for dish_ingredients table
unique_ingredients = set()
# dish_data maps dish_name -> (ingredients list, amounts list) so
# score_all_meals can pass the real ingredients to nutrients_for_dish
dish_data = {}
for category, bucket in raw.items():
if category.startswith("_"):
continue
for dish_name, info in bucket.items():
ingredients = info.get("ingredients", [])
amounts = info.get("amounts_g", [])
kostform = info.get("kostform", "")
dishes.append((dish_name, category, kostform))
```

```
dish_data[dish_name] = (ingredients, amounts)
for ingredient, grams in zip(ingredients, amounts):
dish_ings.append((dish_name, ingredient, float(grams)))
unique_ingredients.add(ingredient)
return dishes, dish_ings, unique_ingredients, dish_data
# Look up real nutrient values for a dish by summing up each ingredient.
# Uses pipeline/frida_matcher.py to translate ingredient names to Frida entries,
# then pipeline/nutrient_lookup.py to get the actual numbers per 100g.
# Ingredients flagged as uncertain by the matcher are skipped.
def nutrients_for_dish(dish_ingredients, dish_amounts, frida_food_df):
profiles = []
for ingredient, grams in zip(dish_ingredients, dish_amounts):
match_result = match_ingredient(ingredient, frida_food_df)
if match_result["flagged_for_review"]:
continue
frida_name = match_result["matched_name"]
try:
profile = nutrients_for_food(frida_name, grams)
profiles.append(profile)
except KeyError:
# Frida name exists in Food sheet but not in Data_Table -- skip
pass
return aggregate_nutrients(profiles) if profiles else {
"energy_kcal": 0.0, "protein_g": 0.0, "fat_sat_g": 0.0,
"fiber_g": 0.0, "sugar_g": 0.0, "sodium_mg": 0.0,
"potassium_mg": 0.0, "calcium_mg": 0.0,
}
# Take a patient's conditions and keep only the ones scoring.py knows about
def conditions_for_scoring(patient):
active = list(patient["conditions"]["primary"]) + list(patient["conditions"]["risk"])
return [c for c in active if c in KNOWN_CONDITIONS]
# Score every patient against every dish and return rows for the meal_scores table
def score_all_meals(dishes, dish_data, patients, frida_food_df):
rows = []
for patient in patients:
conditions = conditions_for_scoring(patient)
for dish_name, category, kostform in dishes:
ingredients, amounts = dish_data.get(dish_name, ([], []))
nutrients = nutrients_for_dish(ingredients, amounts, frida_food_df)
result = score_meal(nutrients, conditions)
rows.append({
"patient_id": patient["id"],
"dish_name": dish_name,
"category": category,
"conditions": ", ".join(conditions),
"score": result["score"],
"grade": result["grade"],
"grade_description": result["grade_description"],
"flag_count": len(result["flags"]),
"energy_kcal": nutrients["energy_kcal"],
"protein_g": nutrients["protein_g"],
"fiber_g": nutrients["fiber_g"],
"calcium_mg": nutrients["calcium_mg"],
"sodium_mg": nutrients["sodium_mg"],
"fat_sat_g": nutrients["fat_sat_g"],
"sugar_g": nutrients["sugar_g"],
})
return rows
# Create the empty tables in the SQLite database
def create_tables(connection):
cursor = connection.cursor()
cursor.executescript("""
CREATE TABLE patients (
id TEXT PRIMARY KEY,
name TEXT NOT NULL,
sex TEXT,
age INTEGER,
location TEXT,
avatar TEXT
);
CREATE TABLE patient_conditions (
```

```
patient_id TEXT NOT NULL,
condition TEXT NOT NULL,
status TEXT NOT NULL CHECK(status IN ('primary','risk','inactive')),
PRIMARY KEY (patient_id, condition, status),
FOREIGN KEY (patient_id) REFERENCES patients(id)
);
CREATE TABLE dishes (
name TEXT PRIMARY KEY,
category TEXT,
kostform TEXT
);
CREATE TABLE dish_ingredients (
dish_name TEXT NOT NULL,
ingredient TEXT NOT NULL,
grams REAL NOT NULL,
PRIMARY KEY (dish_name, ingredient),
FOREIGN KEY (dish_name) REFERENCES dishes(name)
);
CREATE TABLE ingredient_matches (
query TEXT PRIMARY KEY,
matched_name TEXT,
frida_id INTEGER,
confidence REAL,
flagged_for_review INTEGER
);
CREATE TABLE meal_scores (
patient_id TEXT NOT NULL,
dish_name TEXT NOT NULL,
category TEXT,
conditions TEXT,
score REAL,
grade TEXT,
grade_description TEXT,
flag_count INTEGER,
energy_kcal REAL,
protein_g REAL,
fiber_g REAL,
calcium_mg REAL,
sodium_mg REAL,
fat_sat_g REAL,
sugar_g REAL,
PRIMARY KEY (patient_id, dish_name),
FOREIGN KEY (patient_id) REFERENCES patients(id),
FOREIGN KEY (dish_name) REFERENCES dishes(name)
);
""")
# Insert all the patients and their conditions
def insert_patients(connection, patients):
cursor = connection.cursor()
for patient in patients:
cursor.execute(
"INSERT INTO patients VALUES (?, ?, ?, ?, ?, ?)",
(patient["id"], patient["name"], patient["sex"],
patient["age"], patient["location"], patient["avatar"])
)
for status in ("primary", "risk", "inactive"):
for condition in patient["conditions"][status]:
cursor.execute(
"INSERT INTO patient_conditions VALUES (?, ?, ?)",
(patient["id"], condition, status)
)
# Insert dishes and their ingredients
def insert_dishes(connection, dishes, dish_ings):
cursor = connection.cursor()
cursor.executemany("INSERT INTO dishes VALUES (?, ?, ?)", dishes)
cursor.executemany("INSERT INTO dish_ingredients VALUES (?, ?, ?)", dish_ings)
# Insert the Frida matcher results
def insert_matches(connection, matches):
cursor = connection.cursor()
for match in matches.values():
cursor.execute(
```

```
"INSERT INTO ingredient_matches VALUES (?, ?, ?, ?, ?)",
(match["query"], match["matched_name"], match["frida_id"],
match["confidence"], int(match["flagged_for_review"]))
)
# Insert the meal scores
def insert_meal_scores(connection, rows):
cursor = connection.cursor()
for row in rows:
cursor.execute(
"""INSERT INTO meal_scores VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
(row["patient_id"], row["dish_name"], row["category"], row["conditions"],
row["score"], row["grade"], row["grade_description"], row["flag_count"],
row["energy_kcal"], row["protein_g"], row["fiber_g"], row["calcium_mg"],
row["sodium_mg"], row["fat_sat_g"], row["sugar_g"])
)
# Main: run the whole build pipeline
if __name__ == "__main__":
# Step 1: load the Frida Food sheet (used by frida_matcher for name matching)
print("Loading Frida Food sheet...")
frida_food_df = load_frida(str(FRIDA_FILE))
print("Loaded " + str(len(frida_food_df)) + " foods from Frida")
# Step 2: load the Frida Data_Table into the nutrient_lookup cache so it's
# ready for nutrients_for_food() calls inside nutrients_for_dish()
print("Loading Frida Data_Table for nutrient lookup...")
load_frida_dataset()
print("Data_Table loaded")
# Step 3: load all dishes and their ingredients from dish_ingredients.json
print("Loading dishes...")
dishes, dish_ings, unique_ingredients, dish_data = load_dishes(DISH_FILE)
print("Loaded " + str(len(dishes)) + " dishes with " + str(len(unique_ingredients)) + " unique
ingredients")
# Step 4: match every unique ingredient against Frida using the fixed matcher
# (includes manual overrides for ingredients like aeg, loeg, oksekoed etc.)
print("Matching ingredients to Frida...")
matches = {}
for ingredient in sorted(unique_ingredients):
matches[ingredient] = match_ingredient(ingredient, frida_food_df)
flagged_count = sum(1 for m in matches.values() if m["flagged_for_review"])
print("Matched " + str(len(matches)) + " ingredients (" + str(flagged_count) + " flagged for review)")
# Step 5: score every patient against every dish using real Frida nutrients
print("Scoring meals for each patient...")
meal_score_rows = score_all_meals(dishes, dish_data, PATIENTS, frida_food_df)
print("Computed " + str(len(meal_score_rows)) + " patient-meal scores")
# Step 6: write the SQLite database
print("Writing database to " + str(DB_FILE) + " ...")
if DB_FILE.exists():
DB_FILE.unlink()
connection = sqlite3.connect(DB_FILE)
create_tables(connection)
insert_patients(connection, PATIENTS)
insert_dishes(connection, dishes, dish_ings)
insert_matches(connection, matches)
insert_meal_scores(connection, meal_score_rows)
connection.commit()
connection.close()
print("Done. Database is ready at " + str(DB_FILE))
```

api/main.py

```
"""
api/main.py
-----------
Responsibility:
- Expose the NutriScan pipeline as a REST API
- Act as the middle tier between the Dash dashboard (presentation layer)
and the pipeline modules and SQLite database (logic and data layer)
- Validate all requests and responses with Pydantic so the contract
between the dashboard and the pipeline is explicit and documented

Endpoints:
GET /health Liveness check
POST /menu/parse Parse a menu PDF, return raw meal list
POST /score/meal Score one meal for one patient
POST /score/week Score all meals in a menu for one patient
GET /patients Return all patients from the database
GET /patients/{patient_id}/history Return scoring history for one patient

Run the API locally:
uvicorn api.main:app --reload

Auto-generated interactive documentation:
http://127.0.0.1:8000/docs
"""

import sqlite3
import sys
from contextlib import asynccontextmanager
from datetime import datetime
from pathlib import Path
from typing import Optional

# Add project root to path so pipeline imports work when running uvicorn from api/
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# Make sure the pipeline package is importable when running uvicorn from the project root
PROJECT_ROOT = Path(__file__).resolve().parents[1]
DB_FILE = PROJECT_ROOT / "pipeline" / "nutriscan.db"


# Pydantic models
# These describe the shape of every request and response so FastAPI can validate
# inputs, generate the docs page on /docs, and so the dashboard knows exactly
# what to send and expect back.

class ScoreRequest(BaseModel):
pdf_path: str
day_number: int
conditions: list[str]
patient_id: Optional[str] = None


class WeekScoreRequest(BaseModel):
pdf_path: str
conditions: list[str]
patient_id: Optional[str] = None


class MenuParseRequest(BaseModel):
pdf_path: str


class NutrientValues(BaseModel):
energy_kcal: float = 0.0
protein_g: float = 0.0
```

```
fat_sat_g: float = 0.0
fiber_g: float = 0.0
sugar_g: float = 0.0
sodium_mg: float = 0.0
potassium_mg: float = 0.0
calcium_mg: float = 0.0

# potassium_mg is tracked by the pipeline and scoring module (NNR 2023: 3500 mg/day) and is included
here
# so the API contract fully reflects what the pipeline produces.

class MealSummary(BaseModel):
day_number: int
day_name: str
main: str
dessert: str
sides: list[str]
vegetarian: bool


class MealScoreResponse(BaseModel):
day_number: int
day_name: str
main: str
dessert: str
sides: list[str]
vegetarian: bool
grade: str
score: float
grade_description: str
flags: list[str]
nutrients: NutrientValues
coverage_pct: int
unmatched: list[str]
conditions: list[str]


class WeekSummaryResponse(BaseModel):
weekly_grade: str
weekly_avg_score: float
weekly_description: str
grade_counts: dict[str, int]
avg_nutrients: NutrientValues
meal_count: int
all_flags: list[str]
meals: list[MealScoreResponse]


class PatientCondition(BaseModel):
primary: list[str]
risk: list[str]
inactive: list[str]


class PatientResponse(BaseModel):
id: str
name: str
sex: str
age: int
location: str
avatar: str
conditions: PatientCondition


class HistoryEntry(BaseModel):
meal_id: int
grade: str
score: float
scored_at: str
main: str
period: str
```

```
# Lifespan event
# This runs once when the server starts and once when it shuts down.
# We use it to load the Frida dataset into memory so the first /score request
# is fast (otherwise every request would re-read the 12 MB Excel file).

@asynccontextmanager
async def lifespan(app: FastAPI):
# Startup: load Frida once and create the history table if it does not exist
try:
from pipeline.nutrient_lookup import load_frida_dataset
load_frida_dataset()
except Exception as exc:
print(f"[startup] Warning: could not preload Frida dataset: {exc}")

_ensure_history_table()
yield
# Shutdown: nothing to clean up


# Build the FastAPI app with CORS and lifespan attached
app = FastAPI(
title="NutriScan API",
description="REST API for the NutriScan project (Roskilde University, Spring 2026)",
version="1.0",
lifespan=lifespan,
)


# CORS middleware
# The Dash frontend runs on port 8050 and the API on port 8000. Without CORS
# the browser would block every cross-origin request from the dashboard.
app.add_middleware(
CORSMiddleware,
allow_origins=[
"http://localhost:8050",
"http://127.0.0.1:8050",
],
allow_methods=["GET", "POST"],
allow_headers=["*"],
)


# Database helpers

def _get_connection():
"""Opens a SQLite connection with row access by column name."""
if not DB_FILE.exists():
raise HTTPException(
status_code=500,
detail=f"Database not found at {DB_FILE}. Run build_db.py first."
)
connection = sqlite3.connect(DB_FILE)
connection.row_factory = sqlite3.Row
return connection


def _ensure_history_table():
"""Creates the score_history table if it does not exist yet."""
if not DB_FILE.exists():
return

connection = sqlite3.connect(DB_FILE)
connection.execute("""
CREATE TABLE IF NOT EXISTS score_history (
meal_id INTEGER PRIMARY KEY AUTOINCREMENT,
patient_id TEXT NOT NULL,
main TEXT,
period TEXT,
grade TEXT,
```

```
score REAL,
scored_at TEXT NOT NULL,
FOREIGN KEY (patient_id) REFERENCES patients(id)
)
""")
connection.commit()
connection.close()


# Pipeline helpers
# Wrappers around the pipeline modules. Keeping them small means the endpoints
# stay readable and we have one place to handle pipeline errors.

def _resolve_pdf_path(pdf_path: str) -> Path:
"""Resolve the given PDF path relative to the project root if it is not absolute."""
candidate = Path(pdf_path)
if not candidate.is_absolute():
candidate = PROJECT_ROOT / candidate

if not candidate.exists():
raise HTTPException(status_code=404, detail=f"PDF not found: {pdf_path}")
return candidate


def _run_pipeline_extract(pdf_path: str):
"""Calls pdf_extractor.extract_meals() with proper error handling."""
full_path = _resolve_pdf_path(pdf_path)

try:
from pipeline.pdf_extractor import extract_meals
return extract_meals(str(full_path))
except FileNotFoundError as exc:
raise HTTPException(status_code=404, detail=str(exc))
except Exception as exc:
raise HTTPException(
status_code=500,
detail=f"Pipeline failed while extracting meals: {exc}"
)


def _run_menu_parse(pdf_path: str):
"""Calls menu_parser.parse_menu() with proper error handling."""
full_path = _resolve_pdf_path(pdf_path)

try:
from pipeline.menu_parser import parse_menu
return parse_menu(str(full_path))
except Exception as exc:
raise HTTPException(
status_code=500,
detail=f"Pipeline failed while parsing menu: {exc}"
)


def _build_meal_response(meal: dict, score_result: dict, conditions: list[str]) -> dict:
"""Combines the enriched meal and the scoring result into the response shape."""
nutrients = meal.get("nutrients", {})
return {
"day_number": meal.get("day_number", 0),
"day_name": meal.get("day_name", ""),
"main": meal.get("main", ""),
"dessert": meal.get("dessert", ""),
"sides": meal.get("sides", []),
"vegetarian": meal.get("vegetarian", False),
"grade": score_result["grade"],
"score": score_result["score"],
"grade_description": score_result["grade_description"],
"flags": score_result["flags"],
"nutrients": {
"energy_kcal": nutrients.get("energy_kcal", 0.0),
"protein_g": nutrients.get("protein_g", 0.0),
```

```
"fat_sat_g": nutrients.get("fat_sat_g", 0.0),
"fiber_g": nutrients.get("fiber_g", 0.0),
"sugar_g": nutrients.get("sugar_g", 0.0),
"sodium_mg": nutrients.get("sodium_mg", 0.0),
"potassium_mg": nutrients.get("potassium_mg", 0.0),
"calcium_mg": nutrients.get("calcium_mg", 0.0),
},
"coverage_pct": meal.get("coverage_pct", 0),
"unmatched": meal.get("unmatched", []),
"conditions": conditions,
}


def _save_score_to_history(patient_id: str, main: str, period: str,
grade: str, score: float):
"""Inserts one row into the score_history table."""
connection = _get_connection()
connection.execute(
"""INSERT INTO score_history (patient_id, main, period, grade, score, scored_at)
VALUES (?, ?, ?, ?, ?, ?)""",
(patient_id, main, period, grade, score, datetime.utcnow().isoformat())
)
connection.commit()
connection.close()


# 1. Health endpoint

@app.get("/health")
def health():
"""
Liveness check. Returns immediately without touching the pipeline or database.
Used to confirm the server is running.
"""
return {"status": "ok", "version": "1.0"}


# 2. Menu parsing endpoint

@app.post("/menu/parse", response_model=list[MealSummary])
def menu_parse(request: MenuParseRequest):
"""
Parses a menu PDF and returns the raw meal list -- no scoring, no Frida lookup.
Used by the "Meal Plans" page in the dashboard to browse the menu.
"""
raw_meals = _run_menu_parse(request.pdf_path)

return [
{
"day_number": meal.get("day_number", 0),
"day_name": meal.get("day_name", ""),
"main": meal.get("main", ""),
"dessert": meal.get("dessert", ""),
"sides": meal.get("sides", []),
"vegetarian": meal.get("vegetarian", False),
}
for meal in raw_meals
]


# 3. Single-meal scoring endpoint

@app.post("/score/meal", response_model=MealScoreResponse)
def score_meal_endpoint(request: ScoreRequest):
"""
Runs the full pipeline for one day in the menu and returns the grade,
score, flags and nutrients for one patient's conditions.

This replaces the hardcoded preview in the dashboard's "+ ADD Meal" modal.
"""
meals = _run_pipeline_extract(request.pdf_path)
```

```
# Find the requested day
target_meal = None
for meal in meals:
if meal.get("day_number") == request.day_number:
target_meal = meal
break

if target_meal is None:
raise HTTPException(
status_code=404,
detail=f"Day {request.day_number} not found in {request.pdf_path}"
)

# Score the meal
from pipeline.scoring import score_meal as score_meal_fn
result = score_meal_fn(target_meal, request.conditions)
response = _build_meal_response(target_meal, result, request.conditions)

# If a patient_id was provided, record this scoring event in the history table
if request.patient_id:
_save_score_to_history(
patient_id=request.patient_id,
main=target_meal.get("main", ""),
period=Path(request.pdf_path).stem,
grade=result["grade"],
score=result["score"],
)

return response


# 4. Weekly scoring endpoint

@app.post("/score/week", response_model=WeekSummaryResponse)
def score_week_endpoint(request: WeekScoreRequest):
"""
Scores all meals in a PDF for one patient's conditions and returns a
weekly summary plus the individual meal scores.
"""
meals = _run_pipeline_extract(request.pdf_path)

if not meals:
raise HTTPException(
status_code=404,
detail=f"No meals found in {request.pdf_path}"
)

from pipeline.scoring import score_meal as score_meal_fn, score_week

# Score every meal for the patient's conditions
score_results = []
meal_responses = []
for meal in meals:
result = score_meal_fn(meal, request.conditions)
score_results.append(result)
meal_responses.append(_build_meal_response(meal, result, request.conditions))

weekly = score_week(score_results)

# If a patient_id was provided, save every meal to history
if request.patient_id:
period = Path(request.pdf_path).stem
for meal, result in zip(meals, score_results):
_save_score_to_history(
patient_id=request.patient_id,
main=meal.get("main", ""),
period=period,
grade=result["grade"],
score=result["score"],
)
```

```
avg_nutrients = weekly.get("avg_nutrients", {})
return {
"weekly_grade": weekly.get("weekly_grade", "E"),
"weekly_avg_score": weekly.get("weekly_avg_score", 0.0),
"weekly_description": weekly.get("weekly_description", ""),
"grade_counts": weekly.get("grade_counts", {}),
"avg_nutrients": {
"energy_kcal": avg_nutrients.get("energy_kcal", 0.0),
"protein_g": avg_nutrients.get("protein_g", 0.0),
"fat_sat_g": avg_nutrients.get("fat_sat_g", 0.0),
"fiber_g": avg_nutrients.get("fiber_g", 0.0),
"sugar_g": avg_nutrients.get("sugar_g", 0.0),
"sodium_mg": avg_nutrients.get("sodium_mg", 0.0),
"potassium_mg": avg_nutrients.get("potassium_mg", 0.0),
"calcium_mg": avg_nutrients.get("calcium_mg", 0.0),
},
"meal_count": weekly.get("meal_count", 0),
"all_flags": weekly.get("all_flags", []),
"meals": meal_responses,
}


# 5. Patients endpoint

@app.get("/patients", response_model=list[PatientResponse])
def list_patients():
"""
Returns all patients from the SQLite database with their conditions.
Replaces the hardcoded PATIENTS list in dashboard.py.
"""
connection = _get_connection()

patient_rows = connection.execute(
"SELECT id, name, sex, age, location, avatar FROM patients ORDER BY id"
).fetchall()

condition_rows = connection.execute(
"SELECT patient_id, condition, status FROM patient_conditions"
).fetchall()

connection.close()

# Group conditions by patient_id and by status
conditions_by_patient: dict[str, dict[str, list[str]]] = {}
for row in condition_rows:
bucket = conditions_by_patient.setdefault(
row["patient_id"],
{"primary": [], "risk": [], "inactive": []},
)
bucket[row["status"]].append(row["condition"])

result = []
for row in patient_rows:
patient = dict(row)
patient["conditions"] = conditions_by_patient.get(
row["id"],
{"primary": [], "risk": [], "inactive": []},
)
result.append(patient)

return result


# 6. Patient history endpoint

@app.get("/patients/{patient_id}/history", response_model=list[HistoryEntry])
def patient_history(patient_id: str):
"""
Returns all previously saved scores for one patient from the
score_history table. Used by the dashboard to show grades over time.
```

```
"""
connection = _get_connection()

# Check the patient exists
exists = connection.execute(
"SELECT 1 FROM patients WHERE id = ?", (patient_id,)
).fetchone()
if exists is None:
connection.close()
raise HTTPException(status_code=404, detail=f"Patient {patient_id} not found")

rows = connection.execute(
"""SELECT meal_id, grade, score, scored_at, main, period
FROM score_history WHERE patient_id = ?
ORDER BY scored_at DESC""",
(patient_id,),
).fetchall()

connection.close()
return [dict(row) for row in rows]
```

frontend/dashboard.py

```
"""
dashboard.py
------------
Responsibility:
- Provide the web-based frontend for the NutriScan system
- Call the FastAPI backend to score meals and retrieve patient data
- Display nutritional grades, per-nutrient breakdowns, and clinical flags

The dashboard does not contain any scoring or matching logic of its own.
All nutritional computation is delegated to the API, which in turn calls
the pipeline modules (pdf_extractor, frida_matcher, nutrient_lookup, scoring).

This file is structured in six sections:
1. API client functions - thin wrappers around the FastAPI endpoints
2. Fallback data - hardcoded patients and nutrients used only when the API is offline
3. Shared styles - colour constants and CSS dicts reused across components
4. UI components - functions that build one reusable piece of the interface
5. Page layouts - functions that compose components into full pages
6. App layout - the top-level Dash layout (sidebar + content area + modals)
7. Callbacks - functions that make the dashboard interactive

Usage:
Start the API first (terminal 1):
uvicorn api.main:app --reload

Then start the dashboard (terminal 2):
python frontend/dashboard.py

Or start both together from the project root:
python main.py

Open http://127.0.0.1:8050 in your browser.

Dependencies:
pip install dash dash-bootstrap-components requests
"""

import sys
from pathlib import Path

# Make sure the project root is on the path so pipeline imports work
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

import requests
import dash
from dash import html, dcc, Input, Output, State, ctx
import dash_bootstrap_components as dbc


# ----------------
# API Connection
# Tries to fetch live data from the FastAPI backend (port 8000).
# Falls back to hardcoded data if the API is not running, so the dashboard
# still works during development or if something goes wrong on demo day.
# ----------------

API_URL = "http://127.0.0.1:8000"
def api_get_patients():
"""Fetches patients from GET /patients. Returns None if unreachable."""
try:
r = requests.get(f"{API_URL}/patients", timeout=2)
r.raise_for_status()
return r.json()
except Exception:
return None
def api_score_meal(pdf_path, day_number, conditions, patient_id=None):
"""Calls POST /score/meal. Returns None if unreachable."""
try:
r = requests.post(
```

```
f"{API_URL}/score/meal",
json={"pdf_path": pdf_path, "day_number": day_number,
"conditions": conditions, "patient_id": patient_id},
timeout=10,
)
r.raise_for_status()
return r.json()
except Exception:
return None
def api_score_week(pdf_path, conditions, patient_id=None):
"""Calls POST /score/week. Returns None if unreachable."""
try:
r = requests.post(
f"{API_URL}/score/week",
json={"pdf_path": pdf_path, "conditions": conditions, "patient_id": patient_id},
timeout=30,
)
r.raise_for_status()
return r.json()
except Exception:
return None
def api_patient_history(patient_id):
"""Calls GET /patients/{id}/history. Returns empty list if unreachable."""
try:
r = requests.get(f"{API_URL}/patients/{patient_id}/history", timeout=2)
r.raise_for_status()
return r.json()
except Exception:
return []
def api_check_health():
"""Returns True if the API is running, False otherwise."""
try:
r = requests.get(f"{API_URL}/health", timeout=1)
return r.status_code == 200
except Exception:
return False

# -------------------
# Section 1: DATA
# Hardcoded fallback list - used when the API is not working
# Each patient has an id, basic demographics, and a list of health conditions.
# Conditions are split into primary (active), risk (at risk), and inactive.
# -------------------

_PATIENTS_FALLBACK = [
{
"id": "XY-5217",
"name": "Erik Andersen",
"sex": "Male",
"age": 82,
"location": "Wing A, Room 3",
"avatar": "EA",
"conditions": {
"primary": ["Hypertension", "Type 2 Diabetes"],
"risk": ["Malnutrition Risk"],
"inactive": []
}
},
{
"id": "XY-3891",
"name": "Birthe Sørensen",
"sex": "Female",
"age": 79,
"location": "Wing B, Room 7",
"avatar": "BS",
"conditions": {
"primary": ["Osteoporosis"],
"risk": ["Malnutrition Risk"],
"inactive": ["Dementia"]
}
},
```

```
{
"id": "XY-4422",
"name": "Kaj Nielsen",
"sex": "Male",
"age": 85,
"location": "Wing C, Room 12",
"avatar": "KN",
"conditions": {
"primary": ["High Cholesterol"],
"risk": ["Hypertension"],
"inactive": []
}
},
{
"id": "XY-6104",
"name": "Grethe Larsen",
"sex": "Female",
"age": 76,
"location": "Wing A, Room 9",
"avatar": "GL",
"conditions": {
"primary": ["Malnutrition Risk"],
"risk": [],
"inactive": ["Type 2 Diabetes"]
}
},
{
"id": "XY-7733",
"name": "Holger Christensen",
"sex": "Male",
"age": 88,
"location": "Wing D, Room 2",
"avatar": "HC",
"conditions": {
"primary": [],
"risk": [],
"inactive": []
}
}
]

# Try to load live patients from the API on startup; use fallback if it's down.
_live_patients = api_get_patients()
PATIENTS = _live_patients if _live_patients is not None else _PATIENTS_FALLBACK
# Sample meals for the demo.
# In the real system, these come from menu_parser.py reading the Roskilde PDF menus.
# Each meal has a main dish, a list of sides, and a dessert.
SAMPLE_MEALS = {
"March 2026 - Day 1": {
"day": "Sunday 1 March 2026",
"main": {
"name": "Bøf stroganoff",
"ingredients": ["oksekød", "løg", "champignon", "creme fraiche", "tomatpuré"]
},
"sides": ["kartoffelmos", "drueagurker"],
"dessert": {
"name": "Citronfromage",
"ingredients": ["æg", "fløde", "sukker", "citronsaft"]
}
},
"March 2026 - Day 2": {
"day": "Monday 2 March 2026",
"main": {
"name": "Frikadeller",
"ingredients": ["svinekød", "løg", "æg", "mælk", "hvedemel"]
},
"sides": ["stuvet ærter", "gulerødder", "surt"],
"dessert": {
"name": "Cheesecake i glas",
"ingredients": ["flødeost", "fløde", "sukker", "digestive"]
}
```

```
},
"March 2026 - Day 5": {
"day": "Thursday 5 March 2026",
"main": {
"name": "Fiskefrikadeller",
"ingredients": ["torskefilet", "æg", "mælk", "hvedemel", "løg"]
},
"sides": ["caperssovs", "citron", "gulerodsråkost"],
"dessert": {
"name": "Æblegrød",
"ingredients": ["æble", "sukker"]
}
},
"March 2026 - Day 10": {
"day": "Tuesday 10 March 2026",
"main": {
"name": "Hakkebøf",
"ingredients": ["oksekød", "løg", "smør", "æg", "hvedemel"]
},
"sides": ["bløde løg", "skysovs", "rødbeder"],
"dessert": {
"name": "Vanillebudding",
"ingredients": ["mælk", "sukker", "æg", "majsstivelse"]
}
},
"April 2026 - Day 14": {
"day": "Tuesday 14 April 2026",
"main": {
"name": "Frikadeller",
"ingredients": ["svinekød", "løg", "æg", "mælk", "hvedemel"]
},
"sides": ["stuvet ærter", "gulerødder", "surt"],
"dessert": {
"name": "Rabarbergrød",
"ingredients": ["rabarber", "sukker", "kartoffelmel"]
}
}
}
# Nutrient values per meal.
# These are simplified placeholder values for the demo.
# The real pipeline gives much more accurate numbers
# (see pdf_extractor.py + nutrient_lookup.py), but these work fine for the demo.
# Sodium is stored in grams here — calculate_grade() converts to mg before scoring.
# Fallback nutrient values used ONLY when the API is offline.
# When the API is running, all nutrient data comes from the live pipeline instead.
MEAL_NUTRIENTS = {
"March 2026 - Day 1": {"energy": 680, "protein": 38, "carbs": 52, "fibre": 4, "calcium": 180, "sodium":
0.55},
"March 2026 - Day 2": {"energy": 620, "protein": 42, "carbs": 48, "fibre": 6, "calcium": 210, "sodium":
0.45},
"March 2026 - Day 5": {"energy": 520, "protein": 35, "carbs": 40, "fibre": 5, "calcium": 190, "sodium":
0.38},
"March 2026 - Day 10": {"energy": 590, "protein": 36, "carbs": 45, "fibre": 3, "calcium": 160, "
sodium": 0.52},
"April 2026 - Day 14": {"energy": 610, "protein": 40, "carbs": 50, "fibre": 7, "calcium": 220, "
sodium": 0.42},
}
# Condition profiles shown on the Disease Profiles page.
# Each profile has a name, a short description, and a list of nutrient multipliers.
# These multipliers match the ones used in pipeline/scoring.py.
CONDITION_PROFILES = [
{
"name": "Hypertension",
"description": "High blood pressure — affects approx. 2/3 of people over 65",
"multipliers": [
("Sodium", "2.5x penalty", "#d94f3d"),
("Saturated fat", "1.3x penalty", "#e8a838"),
("Fibre", "1.2x reward", "#4caf7d")
]
},
{
```

```
"name": "Type 2 Diabetes",
"description": "Impaired blood sugar regulation — common in nursing home population",
"multipliers": [
("Sugar", "2.5x penalty", "#d94f3d"),
("Energy", "1.3x penalty", "#e8a838"),
("Fibre", "1.5x reward", "#4caf7d")
]
},
{
"name": "Osteoporosis",
"description": "Reduced bone density — elevated fracture risk in elderly",
"multipliers": [
("Calcium", "2.0x reward", "#4caf7d"),
("Protein", "1.3x reward", "#4caf7d")
]
},
{
"name": "Malnutrition Risk",
"description": "Risk of undernutrition — common in elderly with low appetite",
"multipliers": [
("Energy", "0.5x reward (reduced penalty)", "#4caf7d"),
("Protein", "2.0x reward", "#4caf7d")
]
},
{
"name": "High Cholesterol",
"description": "Elevated blood lipids — cardiovascular risk factor",
"multipliers": [
("Saturated fat", "2.0x penalty", "#d94f3d"),
("Fibre", "1.5x reward", "#4caf7d")
]
}
]
# Data sources shown on the Data Sources page.
DATA_SOURCES = [
{
"name": "Nordic Nutrition Recommendations 2023 (NNR 2023)",
"description": "Scientific basis for nutrient reference values across Nordic and Baltic countries.
Primary source for nutrient thresholds used in the scoring algorithm.",
"url": "https://www.norden.org/en/publication/nordic-nutrition-recommendations-2023",
"publisher": "Nordic Council of Ministers"
},
{
"name": "Anbefalinger for den danske institutionskost 2025",
"description": "Official Danish dietary standards for institutional kitchens including nursing homes.
Defines kostformer and daily nutrient targets.",
"url": "https://www.sst.dk",
"publisher": "Fødevarestyrelsen & Sundhedsstyrelsen"
},
{
"name": "Frida Food Data v5.5",
"description": "The Danish national food composition database maintained by DTU. Primary source for all
nutrient values per 100g used in this system.",
"url": "https://frida.fooddata.dk",
"publisher": "DTU National Food Institute"
}
]
# Grade colours used throughout the dashboard.
# Green = good, orange = adequate, red = poor/critical.
GRADE_COLORS = {
"A": "#2d7d46", # forest green
"B": "#5a9e3a", # olive green
"C": "#c8841a", # amber
"D": "#c0532a", # terracotta
"E": "#a82020", # deep red
}

# PDF map — used throughout the dashboard for meal lookups
PDF_MAP = {
"March 2026 — Normal": "sample_data/menuplan-marts-2026.pdf",
"March 2026 — Pescetarisk": "sample_data/pescetarisk-menuplan-vegetarretter-samt-fiskskaldyr.pdf",
```

```
"April 2026 — Normal": "sample_data/menuplan-april-2026.pdf",
"April 2026 — Pescetarisk": "sample_data/menuplan-april-2026-pescetarisk.pdf",
}


def parse_meal_key(meal_key):
"""
Parses a meal key of the form "March 2026 — Normal|1" into
a (pdf_path, day_number) tuple. Falls back gracefully for
the old SAMPLE_MEALS format keys like "March 2026 - Day 1".
"""
if "|" in meal_key:
period, day_str = meal_key.rsplit("|", 1)
pdf_path = PDF_MAP.get(period, "sample_data/menuplan-marts-2026.pdf")
try:
day_number = int(day_str)
except ValueError:
day_number = 1
return pdf_path, day_number

# Legacy format fallback for the five hardcoded SAMPLE_MEALS keys
legacy_map = {
"March 2026": "sample_data/menuplan-marts-2026.pdf",
"April 2026": "sample_data/menuplan-april-2026.pdf",
}
period = meal_key.split(" - ")[0]
pdf_path = legacy_map.get(period, "sample_data/menuplan-marts-2026.pdf")
try:
day_number = int(meal_key.split("Day")[-1].strip())
except (ValueError, IndexError):
day_number = 1
return pdf_path, day_number


def build_meal_dropdown_options():
"""
Fetches all four menus from the API on startup and builds grouped
dropdown options. Each option value is "Period|day_number".
Falls back to the five hardcoded SAMPLE_MEALS if the API is down.
"""
options = []
for period, pdf_path in PDF_MAP.items():
try:
r = requests.post(
f"{API_URL}/menu/parse",
json={"pdf_path": pdf_path},
timeout=15,
)
if r.status_code != 200:
continue
meals = r.json()
options.append({
"label": f"■■ {period} ■■",
"value": f"__header__{period}",
"disabled": True,
})
for meal in meals:
day_num = meal.get("day_number", 0)
main = meal.get("main", "")
options.append({
"label": f"Day {day_num} — {main}",
"value": f"{period}|{day_num}",
})
except Exception:
continue

if not options:
return [{"label": k, "value": k} for k in SAMPLE_MEALS.keys()]

return options
```

```
# Built once at startup — reused by the modal dropdown
MEAL_DROPDOWN_OPTIONS = build_meal_dropdown_options()
# ============================================================================
# SECTION 2: SCORING FUNCTION
# Calls pipeline/scoring.py directly.
# ============================================================================
def calculate_grade(nutrients, conditions):
"""
Wraps pipeline/scoring.py so the dashboard always produces the same
grades as the rest of the system.
The dashboard stores sodium in grams (to match the g display unit),
but scoring.py works in mg — we convert here at the boundary.
"""
try:
from pipeline.scoring import score_meal
except ImportError:
from scoring import score_meal

# Dashboard MEAL_NUTRIENTS uses sodium in grams — convert to mg for scoring.py
nutrients_for_scoring = {
"energy_kcal": nutrients.get("energy", 0),
"protein_g": nutrients.get("protein", 0),
"fiber_g": nutrients.get("fibre", 0),
"calcium_mg": nutrients.get("calcium", 0),
"sodium_mg": nutrients.get("sodium", 0) * 1000, # g → mg
"fat_sat_g": nutrients.get("fat_sat", 0),
"sugar_g": nutrients.get("carbs", 0) * 0.3, # rough: ~30% of carbs as sugar
"potassium_mg": nutrients.get("potassium", 0)
}
result = score_meal(nutrients_for_scoring, conditions)
return result["grade"], result["grade_description"]
# ============================================================================
# SECTION 3: SHARED STYLES
# These style dictionaries are reused across multiple components to keep
# the look consistent without repeating the same CSS everywhere.
# ============================================================================
# Palette as constants so it's easy to tweak in one place
C_SIDEBAR = "#2c3e50" # dark slate
C_ACCENT = "#c8841a" # warm amber
C_BG = "#f5f2ed" # warm off-white
C_CARD = "#ffffff"
C_TEXT = "#2c2c2c"
C_MUTED = "#7a7a7a"
C_BORDER = "#e0dbd3"
C_GREEN = "#2d7d46"
C_RED = "#a82020"
C_ORANGE = "#c8841a"
SIDEBAR_STYLE = {
"width": "230px",
"minHeight": "100vh",
"backgroundColor": C_SIDEBAR,
"position": "fixed",
"top": "0",
"left": "0",
"overflowY": "auto",
"zIndex": "100"
}
CONTENT_STYLE = {
"marginLeft": "230px",
"padding": "28px 32px",
"backgroundColor": C_BG,
"minHeight": "100vh"
}
CARD_STYLE = {
"backgroundColor": C_CARD,
"borderRadius": "10px",
"padding": "20px 24px",
"boxShadow": "0 1px 3px rgba(0,0,0,0.07)",
"marginBottom": "16px",
"border": f"1px solid {C_BORDER}"
```

```
}
# ============================================================================
# SECTION 4: UI Components
# Each function builds one reusable UI component.
# ============================================================================
def make_sidebar():
"""Left navigation bar. Each item has an id like 'nav-dashboard' so the
navigate callback can pick up which one was clicked."""
nav_sections = [
("OVERVIEW", [
("dashboard", "■ Dashboard"),
("patients", "■ Patients"),
("meal-plans", "■■ Meal plans"),
]),
("ANALYSIS", [
("nutritional-gaps", "■ Nutritional gaps"),
("disease-profiles", "■ Disease profiles"),
]),
("DATA", [
("data-sources", "■ Data sources"),
]),
]
nav_items = []
for section_title, links in nav_sections:
nav_items.append(
html.P(section_title, style={
"color": "#8fa3b0",
"fontSize": "12px",
"fontWeight": "bold",
"letterSpacing": "1.5px",
"marginTop": "22px",
"marginBottom": "6px",
"paddingLeft": "18px"
})
)
for page_id, label in links:
nav_items.append(
html.Div(label, id="nav-" + page_id, style={
"padding": "9px 18px",
"borderRadius": "6px",
"cursor": "pointer",
"color": "#c8d6df",
"fontSize": "15px",
"marginBottom": "2px"
})
)
return html.Div([
# Logo
html.Div([
html.H2("NutriScan", style={
"color": C_ACCENT,
"margin": "0",
"fontSize": "22px",
"fontWeight": "bold",
"letterSpacing": "0.5px"
}),
html.P("Meal nutrition analysis · RUC 2026", style={
"color": "#8fa3b0",
"fontSize": "12px",
"margin": "4px 0 0 0"
})
], style={"padding": "24px 18px 14px"}),
html.Hr(style={"borderColor": "#3d5166", "margin": "0 18px"}),
html.Div(nav_items, style={"padding": "6px 8px"}),
# Small disclaimer at the bottom
html.Div([
# API status dot — green when running, grey when offline
html.Div([
html.Span("●", style={
"color": "#2d7d46" if api_check_health() else "#666",
"fontSize": "10px", "marginRight": "5px",
```

```
}),
html.Span(
"API connected" if api_check_health() else "API offline",
style={"fontSize": "10px", "color": "#5a7080"}
),
], style={"marginBottom": "5px"}),
html.P("■ Synthetic patient data.", style={
"fontSize": "10px", "color": "#5a7080", "margin": "0"
}),
html.P("Do not use for clinical decisions.", style={
"fontSize": "10px", "color": "#5a7080", "margin": "2px 0 0 0"
})
], style={"position": "absolute", "bottom": "16px", "padding": "0 18px"})
], style=SIDEBAR_STYLE)
def make_patient_card(patient):
"""Profile banner at the top of the dashboard page."""
badges = []
for condition in patient["conditions"]["primary"]:
badges.append(
html.Span(condition, style={
"backgroundColor": "#fef5e7",
"color": C_ACCENT,
"border": f"1px solid {C_ACCENT}",
"borderRadius": "20px",
"padding": "3px 11px",
"fontSize": "12px",
"fontWeight": "bold",
"marginRight": "6px"
})
)
return html.Div([
html.Div([
# Avatar
html.Div(patient["avatar"], style={
"width": "48px",
"height": "48px",
"borderRadius": "50%",
"backgroundColor": C_ACCENT,
"color": "white",
"display": "flex",
"alignItems": "center",
"justifyContent": "center",
"fontWeight": "bold",
"fontSize": "15px",
"marginRight": "16px",
"flexShrink": "0"
}),
# Info
html.Div([
html.H4(patient["name"], style={
"margin": "0 0 3px 0",
"fontSize": "18px",
"color": C_TEXT
}),
html.P(
f"#{patient['id']} · {patient['sex']} · Age {patient['age']} · {patient['location']}",
style={"margin": "0", "color": C_MUTED, "fontSize": "14px"
}
),
]),
# Condition badges flush right
html.Div(badges, style={
"marginLeft": "auto",
"display": "flex",
"alignItems": "center",
"flexWrap": "wrap",
"gap": "4px"
})
], style={"display": "flex", "alignItems": "center"})
], style={**CARD_STYLE, "marginBottom": "20px",
"borderLeft": f"4px solid {C_ACCENT}"})
```

```
def make_score_cards(nutrients, grade, grade_description):
"""Three summary cards across the top: energy, protein, and grade."""
energy_pct = round(nutrients["energy"] / 700 * 100)
protein_pct = round(nutrients["protein"] / 25 * 100)
grade_color = GRADE_COLORS.get(grade, "#888")
def pct_color(pct):
return C_GREEN if pct >= 85 else (C_ORANGE if pct >= 60 else C_RED)
stat_label_style = {
"fontSize": "10px", "fontWeight": "bold", "letterSpacing": "1px",
"color": C_MUTED, "margin": "0 0 8px 0"
}
stat_value_style = {
"margin": "0 0 4px 0", "fontSize": "28px", "fontWeight": "bold",
"color": C_TEXT
}
stat_sub_style = {
"fontSize": "11px", "color": C_MUTED, "margin": "0 0 6px 0"
}
return html.Div([
# Energy
html.Div([
html.P("ENERGY INTAKE", style=stat_label_style),
html.H3(f"{int(nutrients['energy'])} kcal", style=stat_value_style),
html.P("Target: ~700 kcal / meal", style=stat_sub_style),
html.P(f"{energy_pct}% of target", style={
"fontSize": "12px", "color": pct_color(energy_pct),
"margin": "0", "fontWeight": "bold"
})
], style={**CARD_STYLE, "flex": "1", "marginRight": "12px", "marginBottom": "0"}),
# Protein
html.Div([
html.P("PROTEIN INTAKE", style=stat_label_style),
html.H3(f"{nutrients['protein']} g", style=stat_value_style),
html.P("Target: ~25 g / meal (NNR 2023)", style=stat_sub_style),
html.P(f"{protein_pct}% of target", style={
"fontSize": "12px", "color": pct_color(protein_pct),
"margin": "0", "fontWeight": "bold"
})
], style={**CARD_STYLE, "flex": "1", "marginRight": "12px", "marginBottom": "0"}),
# Grade
html.Div([
html.P("NUTRITIONAL GRADE", style=stat_label_style),
html.Div([
html.Span(grade, style={
"fontSize": "64px",
"fontWeight": "bold",
"color": grade_color,
"lineHeight": "1"
}),
html.Div([
html.P("Grade", style={"fontSize": "14px", "color": C_MUTED, "margin": "0"
}),
html.P(grade_description, style={
"fontSize": "13px", "color": grade_color,
"fontWeight": "bold", "margin": "0"
})
], style={"marginLeft": "14px"})
], style={"display": "flex", "alignItems": "center"}),
html.P("A = Excellent · B = Good · C = Adequate · D = Poor · E = Critical",
style={"fontSize": "9px", "color": "#bbb",
"margin": "10px 0 0 0"
})
], style={**CARD_STYLE, "flex": "1", "marginBottom": "0",
"borderLeft": f"4px solid {grade_color}"})
], style={"display": "flex", "marginBottom": "16px"})
def make_meal_card(meal):
"""Shows today's main dish, sides, and dessert."""
section_label = {
"fontSize": "11px", "fontWeight": "bold", "letterSpacing": "1.5px",
"color": C_MUTED, "margin": "0 0 4px 0"
}
```

```
return html.Div([
html.H5("Today's Meal", style={
"fontWeight": "bold", "marginBottom": "3px",
"color": C_TEXT
}),
html.P(meal["day"], style={
"color": C_MUTED, "fontSize": "12px",
"marginBottom": "14px"
}),
# Main
html.Div([
html.P("MAIN DISH", style=section_label),
html.H5(meal["main"]["name"], style={
"margin": "0 0 3px 0",
"color": C_TEXT
}),
html.P("Ingredients: " + ", ".join(meal["main"]["ingredients"]),
style={"fontSize": "11px", "color": C_MUTED,
"margin": "0"
})
], style={
"backgroundColor": "#fef8ef",
"borderLeft": f"3px solid {C_ACCENT}",
"borderRadius": "0 6px 6px 0",
"padding": "11px 14px",
"marginBottom": "8px"
}),
# Sides
html.Div([
html.P("SIDES", style=section_label),
html.P(", ".join(meal["sides"]),
style={"fontSize": "13px", "margin": "0",
"color": C_TEXT})
], style={
"backgroundColor": "#f7f5f2",
"borderRadius": "6px",
"padding": "11px 14px",
"marginBottom": "8px"
}),
# Dessert
html.Div([
html.P("DESSERT", style=section_label),
html.H5(meal["dessert"]["name"], style={
"margin": "0 0 3px 0",
"color": C_TEXT
}),
html.P("Ingredients: " + ", ".join(meal["dessert"]["ingredients"]),
style={"fontSize": "11px", "color": C_MUTED,
"margin": "0"
})
], style={
"backgroundColor": "#f0f4f8",
"borderLeft": "3px solid #6b8cae",
"borderRadius": "0 6px 6px 0",
"padding": "11px 14px"
}),
], style=CARD_STYLE)
def make_guidelines(nutrients, conditions):
"""
Progress bars for each nutrient with condition-specific alerts.
reverse=True means lower is better (sodium).
"""
nutrient_targets = [
("Energy (kcal)", "energy", 700, "kcal", False),
("Protein (g)", "protein", 25, "g", False),
("Carbohydrate (g)", "carbs", 60, "g", False),
("Dietary fibre (g)", "fibre", 8, "g", False),
("Calcium (mg)", "calcium", 300, "mg", False),
("Salt / sodium (g)", "sodium", 5, "g", True),
]
rows = []
```

```
for label, key, target, unit, reverse in nutrient_targets:
value = nutrients.get(key, 0)
pct = min(round(value / target * 100), 100)
if reverse:
bar_color = C_RED if value >= target else C_GREEN
else:
bar_color = C_GREEN if pct >= 85 else (C_ORANGE if pct >= 60 else C_RED)
alert = None
if reverse and value >= target:
alert = "Exceeds the recommended limit. Check sodium-rich components."
elif not reverse and pct < 60:
alert = "Below target — consider adjusting the meal."
# Condition-specific overrides
if key == "sodium" and "Hypertension" in conditions and value >= target * 0.9:
alert = "■ Hypertension: sodium is approaching the recommended limit for this patient."
if key == "protein" and "Malnutrition Risk" in conditions and pct < 80:
alert = "■ Malnutrition Risk: protein intake is below target for this patient."
if key == "calcium" and "Osteoporosis" in conditions and pct < 80:
alert = "■ Osteoporosis: calcium intake is below the recommended level for bone health."
rows.append(html.Div([
html.Div([
html.Span(label, style={
"fontSize": "12px", "fontWeight": "bold",
"color": C_TEXT
}),
html.Span(f"{value} / {target} {unit}", style={
"fontSize": "11px", "color": C_MUTED, "float": "right"
})
]),
# Bar track
html.Div(
html.Div(style={
"width": f"{pct}%",
"height": "100%",
"backgroundColor": bar_color,
"borderRadius": "4px",
"transition": "width 0.4s ease"
}),
style={
"backgroundColor": "#ede9e3",
"borderRadius": "4px",
"height": "7px",
"margin": "5px 0"
}
),
html.P(alert, style={
"fontSize": "10px",
"color": "white",
"backgroundColor": C_RED,
"borderRadius": "4px",
"padding": "3px 8px",
"margin": "3px 0 10px 0"
}) if alert else html.Div(style={"marginBottom": "10px"})
]))
return html.Div([
html.H5("Nutrient guidelines", style={
"fontWeight": "bold", "marginBottom": "14px",
"color": C_TEXT
}),
html.Div(rows)
], style=CARD_STYLE)
def make_conditions_card(patient):
"""Shows all conditions grouped by type with colour-coded badges."""
badge_colors = {
"primary": {"bg": "#fef5e7", "color": C_ACCENT, "border": C_ACCENT},
"risk": {"bg": "#fdf0ee", "color": "#b03a2e", "border": "#c0532a"},
"inactive": {"bg": "#f5f5f5", "color": "#aaa", "border": "#ddd"}
}
badges = []
for condition_type, condition_list in patient["conditions"].items():
c = badge_colors[condition_type]
```

```
for name in condition_list:
badges.append(html.Div([
html.Span(condition_type.capitalize(), style={
"fontSize": "9px", "color": C_MUTED,
"display": "block"
}),
html.Span(name, style={
"fontSize": "12px", "fontWeight": "bold",
"color": c["color"]
})
], style={
"backgroundColor": c["bg"],
"border": f"1px solid {c['border']}",
"borderRadius": "7px",
"padding": "7px 12px",
"margin": "3px"
}))
if not any(patient["conditions"].values()):
badges.append(html.P("No active conditions", style={
"color": C_MUTED, "fontSize": "13px",
"padding": "6px"
}))
return html.Div([
html.H5("Active conditions", style={
"fontWeight": "bold", "marginBottom": "10px",
"color": C_TEXT
}),
html.Div(badges, style={"display": "flex", "flexWrap": "wrap"})
], style=CARD_STYLE)
# ================================================================================
# SECTION 5: PAGE LAYOUTS
# Each function builds a full page by combining the helper components above.
# ================================================================================

def page_dashboard(patient, meal_key, api_score=None):
all_conditions = patient["conditions"]["primary"] + patient["conditions"]["risk"]

# Try to get the meal from SAMPLE_MEALS (old format)
# If the key is new format "Period|day", fall back gracefully
meal = SAMPLE_MEALS.get(meal_key)
nutrients = MEAL_NUTRIENTS.get(meal_key, {})

# Use the real API score if we have one
if api_score:
grade = api_score["grade"]
grade_description = api_score["grade_description"]
api_nutrients = api_score.get("nutrients")
if api_nutrients:
nutrients = {
"energy": round(api_nutrients.get("energy_kcal", nutrients.get("energy", 0)), 0),
"protein": round(api_nutrients.get("protein_g", nutrients.get("protein", 0)), 1),
"carbs": round(api_nutrients.get("sugar_g", 0) / 0.3, 1), # reverse the ~30% estimate
"fibre": round(api_nutrients.get("fiber_g", nutrients.get("fibre", 0)), 1),
"calcium": round(api_nutrients.get("calcium_mg", nutrients.get("calcium", 0)), 0),
"sodium": round(api_nutrients.get("sodium_mg", 0) / 1000, 2),
}
else:
if nutrients:
grade, grade_description = calculate_grade(nutrients, all_conditions)
else:
grade, grade_description = "?", "Select a meal to see a score"

# If no meal in SAMPLE_MEALS, build a minimal display from the meal_key and api_score
if not meal:
if api_score:
main_name = api_score.get("main", meal_key)
sides = api_score.get("sides", [])
dessert = api_score.get("dessert", "")
day_name = api_score.get("day_name", "")
else:
# Parse what we can from the key
```

```
if "|" in meal_key:
period, day_str = meal_key.rsplit("|", 1)
main_name = f"Day {day_str} — {period}"
else:
main_name = meal_key
sides, dessert, day_name = [], "", ""

meal = {
"day": day_name,
"main": {"name": main_name, "ingredients": []},
"sides": sides,
"dessert": {"name": dessert, "ingredients": []},
}

# Ensure nutrients has all keys make_score_cards and make_guidelines expect
nutrients.setdefault("energy", 0)
nutrients.setdefault("protein", 0)
nutrients.setdefault("carbs", 0)
nutrients.setdefault("fibre", 0)
nutrients.setdefault("calcium", 0)
nutrients.setdefault("sodium", 0)

return html.Div([
make_patient_card(patient),
make_score_cards(nutrients, grade, grade_description),
html.Div([
html.Div([
make_conditions_card(patient),
make_meal_card(meal),
], style={"width": "45%", "paddingRight": "12px"}),
html.Div([
make_guidelines(nutrients, all_conditions),
], style={"width": "55%"}),
], style={"display": "flex"})
])
def page_patients():
"""
Builds the patients list page showing all synthetic patients.
Each patient has a 'View Dashboard' button that navigates to their dashboard.
"""
patient_cards = []
cards = []
for patient in PATIENTS:
all_conditions = patient["conditions"]["primary"] + patient["conditions"]["risk"]
condition_text = ", ".join(all_conditions) if all_conditions else "No active conditions"
cards.append(html.Div([
html.Div([
html.Div(patient["avatar"], style={
"width": "42px", "height": "42px", "borderRadius": "50%",
"backgroundColor": C_ACCENT, "color": "white",
"display": "flex", "alignItems": "center",
"justifyContent": "center", "fontWeight": "bold",
"fontSize": "13px", "marginRight": "14px", "flexShrink": "0"
}),
html.Div([
html.H5(patient["name"], style={
"margin": "0 0 2px 0",
"color": C_TEXT
}),
html.P(
f"#{patient['id']} · {patient['sex']} · Age {patient['age']} · {patient['location']}",
style={"margin": "0", "color": C_MUTED,
"fontSize": "12px"
}
),
html.P(condition_text, style={
"margin": "3px 0 0 0", "fontSize": "12px",
"color": C_ACCENT, "fontWeight": "bold"
})
]),
html.Button("View dashboard", id="view-patient-" + patient["id"], style={
```

```
"marginLeft": "auto",
"backgroundColor": C_ACCENT,
"border": "none",
"borderRadius": "6px",
"padding": "7px 14px",
"color": "white",
"cursor": "pointer",
"fontWeight": "bold",
"fontSize": "12px"
})
], style={"display": "flex", "alignItems": "center"})
], style=CARD_STYLE))
return html.Div([
html.H4("Patients", style={
"fontWeight": "bold", "marginBottom": "6px",
"color": C_TEXT
}),
html.P(
f"{len(PATIENTS)} synthetic patients — for testing and demonstration only.",
style={"color": C_MUTED, "fontSize": "13px",
"marginBottom": "20px"
}
),
html.Div(cards)
])
def page_disease_profiles():
"""
Builds the disease profiles page explaining the condition multipliers
used in the scoring algorithm.
"""
profile_cards = []
for profile in CONDITION_PROFILES:
rows = []
for nutrient_name, effect_text, effect_color in profile["multipliers"]:
rows.append(html.Div([
html.Span(nutrient_name, style={
"fontSize": "13px", "fontWeight": "bold",
"flex": "1",
"color": C_TEXT
}),
html.Span(effect_text, style={
"fontSize": "11px",
"color": "white",
"backgroundColor": effect_color,
"borderRadius": "12px",
"padding": "2px 10px"
})
], style={
"display": "flex",
"alignItems": "center",
"justifyContent": "space-between",
"padding": "7px 0",
"borderBottom": f"1px solid {C_BORDER}"
}))
profile_cards.append(html.Div([
html.H5(profile["name"], style={
"fontWeight": "bold", "marginBottom": "3px",
"color": C_TEXT
}),
html.P(profile["description"], style={
"color": C_MUTED, "fontSize": "12px",
"marginBottom": "10px"
}),
html.Div(rows)
], style=CARD_STYLE))
return html.Div([
html.H4("Disease profiles", style={
"fontWeight": "bold", "marginBottom": "6px",
"color": C_TEXT
}),
html.P(
```

```
"How each condition changes the scoring weights. "
"Based on NNR 2023 and Institutionskost 2025.",
style={"color": C_MUTED, "fontSize": "13px",
"marginBottom": "20px"
}
),
html.Div(profile_cards)
])
def page_data_sources():
"""
Builds the data sources page with links to NNR 2023, Institutionskost, and Frida.
"""
source_cards = []
for source in DATA_SOURCES:
source_cards.append(html.Div([
html.H5(source["name"], style={"fontWeight": "700", "marginBottom": "4px"}),
html.P(source["publisher"], style={
"color": "#e8a838", "fontSize": "12px",
"fontWeight": "600", "marginBottom": "8px"
}),
html.P(source["description"], style={
"color": "#666", "fontSize": "13px", "marginBottom": "12px"
}),
html.A("Visit source →", href=source["url"], target="_blank", style={
"color": "#e8a838", "fontSize": "13px", "fontWeight": "600"
})
], style=CARD_STYLE))
return html.Div([
html.H4("Data Sources", style={"fontWeight": "700", "marginBottom": "8px"}),
html.P(
"All nutritional guidelines and food composition data used in this system.",
style={"color": "#666", "fontSize": "13px", "marginBottom": "20px"}
),
html.Div(source_cards)
])

def page_meal_plans():
"""
Fetches and displays the parsed meal list for all four menu PDFs.
Calls POST /menu/parse on the API for each menu.
Falls back to a message if the API is offline.
"""
menus = [
("March 2026 — Normal", "sample_data/menuplan-marts-2026.pdf"),
("March 2026 — Pescetarisk", "sample_data/pescetarisk-menuplan-vegetarretter-samt-fiskskaldyr.pdf"),
("April 2026 — Normal", "sample_data/menuplan-april-2026.pdf"),
("April 2026 — Pescetarisk", "sample_data/menuplan-april-2026-pescetarisk.pdf"),
]

# Try to fetch each menu from the API
fetched = {}
for title, pdf_path in menus:
try:
r = requests.post(
f"{API_URL}/menu/parse",
json={"pdf_path": pdf_path},
timeout=15,
)
if r.status_code == 200:
fetched[title] = r.json()
except Exception:
pass

if not fetched:
return page_placeholder(
"Meal Plans",
"Could not load menus — make sure the API is running."
)

def make_menu_section(title, meals):
if not meals:
```

```
return html.Div()

rows = []
for meal in meals:
veg_badge = html.Span("Vegetarisk", style={
"backgroundColor": "#e8f5e9",
"color": "#2d7d46",
"border": "1px solid #2d7d46",
"borderRadius": "12px",
"padding": "1px 8px",
"fontSize": "10px",
"fontWeight": "bold",
"marginLeft": "8px",
}) if meal.get("vegetarian") else None

rows.append(html.Div([
# Day number badge
html.Div(str(meal.get("day_number", "")), style={
"width": "32px",
"height": "32px",
"borderRadius": "50%",
"backgroundColor": C_ACCENT,
"color": "white",
"display": "flex",
"alignItems": "center",
"justifyContent": "center",
"fontWeight": "bold",
"fontSize": "12px",
"flexShrink": "0",
"marginRight": "14px",
}),
# Meal info
html.Div([
html.Div([
html.Span(meal.get("main", ""), style={
"fontWeight": "bold",
"fontSize": "14px",
"color": C_TEXT,
}),
veg_badge,
], style={"display": "flex", "alignItems": "center",
"marginBottom": "2px"}),
html.P(
"Sides: " + ", ".join(meal.get("sides", [])) if meal.get("sides") else "",
style={"margin": "0", "color": C_MUTED, "fontSize": "11px"}
),
html.P(
"Dessert: " + meal.get("dessert", ""),
style={"margin": "0", "color": C_MUTED, "fontSize": "11px"}
),
], style={"flex": "1"}),
# Day name on the right
html.Span(meal.get("day_name", ""), style={
"color": C_MUTED,
"fontSize": "12px",
"alignSelf": "center",
}),
], style={
"display": "flex",
"alignItems": "flex-start",
"padding": "12px 0",
"borderBottom": f"1px solid {C_BORDER}",
}))

return html.Div([
html.H5(title, style={
"fontWeight": "bold", "marginBottom": "4px", "color": C_TEXT,
}),
html.P(f"{len(meals)} meals", style={
"color": C_MUTED, "fontSize": "12px", "marginBottom": "14px",
}),
```

```
html.Div(rows),
], style=CARD_STYLE)

return html.Div([
html.H4("Meal plans", style={
"fontWeight": "bold", "marginBottom": "6px", "color": C_TEXT,
}),
html.P(
"Full menus from Madservice Roskilde Kommune, parsed directly from the PDF.",
style={"color": C_MUTED, "fontSize": "13px", "marginBottom": "20px"}
),
*[make_menu_section(title, meals) for title, meals in fetched.items()]
])

def page_nutritional_gaps(patient, pdf_path="sample_data/menuplan-marts-2026.pdf"):
"""
Fetches a full weekly score from the API for the selected patient
and displays grade distribution, average nutrients, and grouped clinical flags.
"""
conditions = patient["conditions"]["primary"] + patient["conditions"]["risk"]
weekly = api_score_week(pdf_path, conditions, patient["id"])
if not weekly:
return page_placeholder(
"Nutritional Gaps",
"Could not load weekly analysis — make sure the API is running."
)
total_meals = weekly.get("meal_count", 1)
grade_counts = weekly.get("grade_counts", {})
avg = weekly.get("avg_nutrients", {})
all_flags = list(dict.fromkeys(weekly.get("all_flags", [])))
# Grade distribution bar
grade_bar_items = []
for grade in ["A", "B", "C", "D", "E"]:
count = grade_counts.get(grade, 0)
if count == 0:
continue
grade_bar_items.append(
html.Div(
f"{grade} {count}",
style={
"flex": str(count),
"backgroundColor": GRADE_COLORS.get(grade, "#888"),
"color": "white",
"fontSize": "13px",
"fontWeight": "bold",
"display": "flex",
"alignItems": "center",
"justifyContent": "center",
"minWidth": "24px",
}
)
)
# Average nutrient bars
avg_rows = [
("Energy", f"{round(avg.get('energy_kcal', 0))} kcal", avg.get("energy_kcal", 0) / 700),
("Protein", f"{round(avg.get('protein_g', 0), 1)} g", avg.get("protein_g", 0) / 25),
("Dietary fibre", f"{round(avg.get('fiber_g', 0), 1)} g", avg.get("fiber_g", 0) / 8),
("Calcium", f"{round(avg.get('calcium_mg', 0))} mg", avg.get("calcium_mg", 0) / 300),
("Sodium", f"{round(avg.get('sodium_mg', 0) / 1000, 2)} g", min((avg.get("sodium_mg", 0) / 1000) / 5,
1.0)),
]
nutrient_rows = []
for label, value_str, ratio in avg_rows:
capped = min(ratio, 1.0)
color = C_GREEN if capped >= 0.85 else (C_ORANGE if capped >= 0.6 else C_RED)
nutrient_rows.append(html.Div([
html.Div([
html.Span(label, style={"fontSize": "14px", "fontWeight": "bold", "color": C_TEXT}),
html.Span(value_str, style={"fontSize": "12px", "color": C_MUTED, "float": "right"}),
]),
html.Div(
```

```
html.Div(style={
"width": f"{round(capped * 100)}%",
"height": "100%",
"backgroundColor": color,
"borderRadius": "4px",
}),
style={"backgroundColor": "#ede9e3", "borderRadius": "4px",
"height": "7px", "margin": "5px 0"}
),
], style={"marginBottom": "10px"}))
# Grouped flags with reframed language
from collections import Counter
flag_summary = Counter()
for flag in all_flags:
if "Sodium" in flag:
flag_summary["Sodium above recommended limit"] += 1
elif "exceeds upper limit" in flag:
flag_summary["Energy above recommended range"] += 1
elif "Energy" in flag and "below" in flag:
flag_summary["Energy below recommended range"] += 1
elif "Calcium" in flag:
flag_summary["Calcium below target"] += 1
elif "Protein" in flag:
flag_summary["Protein below target"] += 1
elif "Fibre" in flag:
flag_summary["Fibre below target"] += 1
elif "Saturated fat" in flag:
flag_summary["Saturated fat above recommended limit"] += 1
elif "Sugar" in flag:
flag_summary["Sugar above recommended limit"] += 1
flag_items = [
html.Div([
html.Span(label, style={
"fontSize": "12px", "fontWeight": "bold",
"color": C_RED, "flex": "1",
}),
html.Span(
f"{count} of {total_meals} meals",
style={"fontSize": "11px", "color": C_MUTED}
),
], style={
"display": "flex",
"justifyContent": "space-between",
"alignItems": "center",
"backgroundColor": "#fdf0ee",
"border": f"1px solid {C_RED}",
"borderRadius": "6px",
"padding": "8px 12px",
"marginBottom": "6px",
})
for label, count in sorted(flag_summary.items(), key=lambda x: -x[1])
] if flag_summary else [
html.P("No nutrient concerns identified this month.", style={
"color": C_MUTED, "fontSize": "13px"
})
]
return html.Div([
html.H4("Nutritional gaps", style={
"fontWeight": "bold", "marginBottom": "6px", "color": C_TEXT,
}),
html.P(
f"How well the March 2026 menu meets the nutritional needs of "
f"{patient['name']} across all {total_meals} meals. "
f"Average grade: {weekly.get('weekly_grade', '---')} - "
f"{weekly.get('weekly_description', '')}.",
style={"color": C_MUTED, "fontSize": "13px", "marginBottom": "4px"}
),
html.P(
"Note: flags are raised when a single meal falls outside the ranges "
"recommended by NNR 2023. They do not account for snacks, drinks, or "
"other meals during the day and are intended as a screening tool for "
```

```
"care staff, not a verdict on overall diet quality. Sugar flags are primarily driven by dessert
ingredients and reflect a known characteristic of traditional Danish institutional menus.",
style={
"color": C_MUTED, "fontSize": "11px",
"fontStyle": "italic", "marginBottom": "20px",
}
),
# Grade distribution
html.Div([
html.H5("Grade distribution", style={
"fontWeight": "bold", "marginBottom": "10px", "color": C_TEXT,
}),
html.Div(grade_bar_items, style={
"display": "flex",
"height": "32px",
"borderRadius": "6px",
"overflow": "hidden",
"marginBottom": "8px",
}),
html.P(
" . ".join([
f"{g}: {grade_counts.get(g, 0)}"
for g in ["A", "B", "C", "D", "E"]
]),
style={"fontSize": "11px", "color": C_MUTED, "margin": "0"}
),
], style=CARD_STYLE),
# Average nutrients
html.Div([
html.H5("Average nutrients per meal", style={
"fontWeight": "bold", "marginBottom": "14px", "color": C_TEXT,
}),
html.Div(nutrient_rows),
], style=CARD_STYLE),
# Areas to review
html.Div([
html.H5("Areas to review", style={
"fontWeight": "bold", "marginBottom": "4px", "color": C_TEXT,
}),
html.P(
"Nutrients outside the recommended range in one or more meals. "
"Intended as a prompt for dietician review, not a clinical diagnosis.",
style={"color": C_MUTED, "fontSize": "11px", "marginBottom": "12px"}
),
html.Div(flag_items),
], style=CARD_STYLE),
])
def page_placeholder(title, message):
"""
A simple placeholder page for sections that are not yet implemented.
"""
return html.Div([
html.H4(title, style={
"fontWeight": "bold", "marginBottom": "8px",
"color": C_TEXT
}),
html.Div([
html.P("■", style={"fontSize": "44px", "margin": "0 0 12px 0"}),
html.P(message, style={
"color": C_MUTED, "fontSize": "14px"
})
], style={**CARD_STYLE, "textAlign": "center", "padding": "56px 20px"})
])

# ================================================================================
# SECTION 6: ADD MEAL MODAL
# The modal is the popup window that appears when "+ ADD Meal" is clicked.
# It lets the user select a patient and a menu day, shows a preview of the
# score, and then updates the dashboard when confirmed.
# ================================================================================
def make_welcome_modal():
```

```
nav_rows = [
("■ Dashboard", "Your main view. Select a patient and score a meal to see the nutritional grade and
clinical flags."),
("■ Patients", "Browse all residents and their health conditions at a glance."),
("■■ Meal plans", "View the full monthly menu from Madservice Roskilde."),
("■ Nutritional gaps", "Week-level summary of average nutrient intake for a resident."),
("■ Disease profiles", "See how each health condition adjusts the scoring multipliers."),
("■ Data sources", "Information about the Frida 5.5 database and menu data used."),
]
nav_items = []
for icon_label, desc in nav_rows:
nav_items.append(
html.Div([
html.Span(icon_label, style={"fontWeight": "bold", "color": C_TEXT, "fontSize": "13px", "minWidth": "
170px", "display": "inline-block"}),
html.Span(desc, style={"color": C_MUTED, "fontSize": "13px"}),
], style={"marginBottom": "8px", "display": "flex", "gap": "12px"})
)
return dbc.Modal([
dbc.ModalHeader(dbc.ModalTitle("Welcome to NutriScan", style={"color": C_TEXT, "fontSize": "18px"}),
close_button=False),
dbc.ModalBody([
html.P("NutriScan scores institutional meals against NNR 2023 and Institutionskost 2025, personalised
to each resident's health conditions.", style={"color": C_MUTED, "fontSize": "13px", "marginBottom": "
18px"}),
html.P("HOW TO SCORE A MEAL", style={"fontSize": "10px", "fontWeight": "bold", "letterSpacing": "
1.5px", "color": C_MUTED, "marginBottom": "10px"}),
html.Div([
html.Div([
html.Div("1", style={"width": "26px", "height": "26px", "borderRadius": "50%", "backgroundColor":
C_ACCENT, "color": "white", "display": "flex", "alignItems": "center", "justifyContent": "center", "
fontWeight": "bold", "fontSize": "13px", "flexShrink": "0"}),
html.Div([html.Span("Click "), html.Span("+ Add meal", style={"backgroundColor": C_ACCENT, "color": "
white", "borderRadius": "4px", "padding": "2px 8px", "fontSize": "12px", "fontWeight": "bold"}),
html.Span(" in the top-right corner.")]),
], style={"display": "flex", "alignItems": "center", "gap": "12px", "marginBottom": "10px"}),
html.Div([
html.Div("2", style={"width": "26px", "height": "26px", "borderRadius": "50%", "backgroundColor":
C_ACCENT, "color": "white", "display": "flex", "alignItems": "center", "justifyContent": "center", "
fontWeight": "bold", "fontSize": "13px", "flexShrink": "0"}),
html.Span("Select a resident from the dropdown.", style={"color": C_TEXT, "fontSize": "13px"}),
], style={"display": "flex", "alignItems": "center", "gap": "12px", "marginBottom": "10px"}),
html.Div([
html.Div("3", style={"width": "26px", "height": "26px", "borderRadius": "50%", "backgroundColor":
C_ACCENT, "color": "white", "display": "flex", "alignItems": "center", "justifyContent": "center", "
fontWeight": "bold", "fontSize": "13px", "flexShrink": "0"}),
html.Span("Pick a meal day from one of the four menu dropdowns, then click Score this meal.",
style={"color": C_TEXT, "fontSize": "13px"}),
], style={"display": "flex", "alignItems": "center", "gap": "12px", "marginBottom": "18px"}),
]),
html.Hr(style={"borderColor": C_BORDER}),
html.P("PAGES", style={"fontSize": "10px", "fontWeight": "bold", "letterSpacing": "1.5px", "color":
C_MUTED, "marginBottom": "10px", "marginTop": "14px"}),
html.Div(nav_items),
html.Hr(style={"borderColor": C_BORDER}),
html.P("■ NutriScan uses synthetic patient data. Do not use for clinical decisions.", style={"color":
C_MUTED, "fontSize": "11px", "marginTop": "12px", "marginBottom": "0"}),
]),
dbc.ModalFooter(
html.Button("Got it — take me to the dashboard", id="welcome-close", style={"backgroundColor":
C_ACCENT, "border": "none", "borderRadius": "6px", "padding": "8px 20px", "color": "white", "cursor": "
pointer", "fontWeight": "bold", "fontSize": "13px"})
)
], id="welcome-modal", is_open=True, size="lg", backdrop="static")


def make_add_meal_modal():
"""
Builds the modal popup for scoring a meal.
Four separate dropdowns — one per menu type — replace the single
grouped dropdown so it's clearer which menu you're selecting from.
```

```
"""
patient_options = [
{"label": f"{p['name']} (#{p['id']})", "value": p["id"]}
for p in PATIENTS
]

# Build options per menu from MEAL_DROPDOWN_OPTIONS
# MEAL_DROPDOWN_OPTIONS entries are either headers (disabled) or
# "Period|day_number" values — split them by period
menus_options = {
"March 2026 — Normal": [],
"March 2026 — Pescetarisk": [],
"April 2026 — Normal": [],
"April 2026 — Pescetarisk": [],
}
for opt in MEAL_DROPDOWN_OPTIONS:
if opt.get("disabled"):
continue
value = opt["value"]
if "|" in value:
period, _ = value.rsplit("|", 1)
if period in menus_options:
menus_options[period].append(opt)

# If API was down and we got legacy SAMPLE_MEALS fallback, put them all under March Normal
if not any(menus_options.values()):
menus_options["March 2026 — Normal"] = MEAL_DROPDOWN_OPTIONS

menu_labels = {
"March 2026 — Normal": "March 2026 — Normal",
"March 2026 — Pescetarisk": "March 2026 — Pescetarisk",
"April 2026 — Normal": "April 2026 — Normal",
"April 2026 — Pescetarisk": "April 2026 — Pescetarisk",
}

menu_dropdowns = []
for period, options in menus_options.items():
dropdown_id = "modal-meal-" + period.lower().replace(" ", "-").replace("—", "").replace("--", "-")
menu_dropdowns.append(
html.Div([
html.Label(menu_labels[period], style={
"fontWeight": "bold",
"fontSize": "12px",
"color": C_MUTED,
"marginBottom": "4px",
"display": "block",
}),
dcc.Dropdown(
id={"type": "menu-dropdown", "period": period},
options=options,
placeholder=f"Choose a day from {period}...",
style={"marginBottom": "10px"},
maxHeight=300,
)
])
)

return dbc.Modal([
dbc.ModalHeader(dbc.ModalTitle("Score a meal", style={"color": C_TEXT})),
dbc.ModalBody([
html.P(
"Pick a patient and a menu day. The system will run the scoring algorithm "
"and show you a grade preview before you confirm.",
style={"color": C_MUTED, "marginBottom": "18px", "fontSize": "15px"}
),
html.Label("Step 1: Select a resident", style={"fontWeight": "bold", "marginBottom": "5px", "color": "
#5a5a5a"}),
dcc.Dropdown(
id="modal-patient-select",
options=patient_options,
placeholder="Choose a patient...",
```

```
style={"marginBottom": "18px"},
maxHeight=200,
),
html.Label("Step 2: Select a meal day", style={"fontWeight": "bold", "marginBottom": "15px", "display": "
block", "color": "#5a5a5a"}),
html.Div(menu_dropdowns),
# Hidden single dropdown that callbacks read from — set by the pattern-match callback below
dcc.Dropdown(
id="modal-meal-select",
options=[],
value=None,
style={"display": "none"},
),
html.Div(id="modal-preview", style={"marginTop": "14px"})
]),
dbc.ModalFooter([
html.Button("Cancel", id="modal-cancel", style={
"backgroundColor": "white",
"border": f"1px solid {C_BORDER}",
"borderRadius": "6px",
"padding": "7px 18px",
"marginRight": "8px",
"cursor": "pointer"
}),
html.Button("Step 3: Score this meal →", id="modal-score", style={
"backgroundColor": C_ACCENT,
"border": "none",
"borderRadius": "6px",
"padding": "7px 18px",
"color": "white",
"cursor": "pointer",
"fontWeight": "bold"
})
])
], id="add-meal-modal", is_open=False, size="lg")
# ================================================================================
# SECTION 7: APP LAYOUT
# This puts together the full page structure: sidebar + main content area.
# dcc.Store is used to remember the current page and selected patient/meal
# across interactions without needing a database.
# ================================================================================
app = dash.Dash(
__name__,
external_stylesheets=[dbc.themes.BOOTSTRAP],
title="NutriScan",
suppress_callback_exceptions=True
)
app.layout = html.Div([
# These stores act like variables that persist across interactions.
# They store the current page name and the selected patient/meal.
dcc.Store(id="current-page", data="dashboard"),
dcc.Store(id="selected-patient-id", data=PATIENTS[0]["id"]),
dcc.Store(id="selected-meal-key", data=list(SAMPLE_MEALS.keys())[0]),
dcc.Store(id="last-api-score", data=None),

# Sidebar (always visible on the left)
make_sidebar(),
# Modal popup (hidden by default, shown when + ADD Meal is clicked)
make_welcome_modal(),
make_add_meal_modal(),
# Main content area
html.Div([
# Top bar with title and buttons
html.Div([
html.H3("Patient Nutrition Dashboard", id="page-title", style={
"margin": "0", "fontWeight": "bold",
"color": C_TEXT,
"fontSize": "20px"
}),
html.Div([
html.Button("■ Export PDF", id="export-pdf-btn", style={
```

```
"backgroundColor": "white",
"border": f"1px solid {C_BORDER}",
"borderRadius": "6px",
"padding": "7px 14px",
"marginRight": "10px",
"cursor": "pointer",
"fontWeight": "bold",
"fontSize": "12px",
"color": C_TEXT
}),
html.Button("+ Add meal", id="open-modal-btn", style={
"backgroundColor": C_ACCENT,
"border": "none",
"borderRadius": "6px",
"padding": "7px 14px",
"color": "white",
"cursor": "pointer",
"fontWeight": "bold",
"fontSize": "12px"
})
])
], style={
"display": "flex",
"justifyContent": "space-between",
"alignItems": "center",
"marginBottom": "22px",
"paddingBottom": "14px",
"borderBottom": f"1px solid {C_BORDER}"
}),
html.Div(id="page-content")
], style=CONTENT_STYLE)
])

# ================================================================================
# SECTION 8: CALLBACKS
# Callbacks are what make Dash interactive. Each callback is a function that
# runs automatically when its Input changes, and updates its Output.
# ================================================================================
@app.callback(
Output("welcome-modal", "is_open"),
Input("welcome-close", "n_clicks"),
prevent_initial_call=True
)
def close_welcome_modal(n):
return False

# --- Callback 1: Render the correct page ---
# This runs whenever the current page, patient, or meal changes.
# It calls the API directly so the score is always fresh and synchronous —
# no race condition with a separate auto-score callback.
@app.callback(
Output("page-content", "children"),
Output("page-title", "children"),
Input("current-page", "data"),
Input("selected-patient-id", "data"),
Input("selected-meal-key", "data"),
)
def render_page(page, patient_id, meal_key):
selected_patient = next(
(p for p in PATIENTS if p["id"] == patient_id), PATIENTS[0]
)
page_titles = {
"dashboard": "Patient Nutrition Dashboard",
"patients": "Patients",
"meal-plans": "Meal Plans",
"nutritional-gaps": "Nutritional Gaps",
"frida-foods": "Frida Foods",
"disease-profiles": "Disease Profiles",
"data-sources": "Data Sources",
}
title = page_titles.get(page, "NutriScan")
```

```
# For the dashboard page, fetch a live score from the API right now.
# This makes the render synchronous — no separate auto-score callback needed.
api_score = None
if page == "dashboard" and meal_key:
conditions = (
selected_patient["conditions"]["primary"] +
selected_patient["conditions"]["risk"]
)
pdf_path, day_number = parse_meal_key(meal_key)
api_score = api_score_meal(pdf_path, day_number, conditions, selected_patient["id"])
page_map = {
"dashboard": lambda: page_dashboard(selected_patient, meal_key, api_score),
"patients": page_patients,
"disease-profiles": page_disease_profiles,
"data-sources": page_data_sources,
"nutritional-gaps": lambda: page_nutritional_gaps(selected_patient),
"settings": lambda: page_placeholder("Settings", "Reserved for future configuration options."),
"meal-plans": page_meal_plans,
}
content_fn = page_map.get(page, lambda: page_dashboard(selected_patient, meal_key, api_score))
return content_fn(), title
# --- Callback 2: Sidebar navigation ---
# This runs when any sidebar link is clicked.
# It updates the current-page store, which triggers callback 1 above.
@app.callback(
Output("current-page", "data"),
[Input("nav-" + page, "n_clicks") for page in [
"dashboard", "patients", "meal-plans", "nutritional-gaps",
"disease-profiles", "data-sources"
]],
prevent_initial_call=True
)

def navigate(*args):
triggered = ctx.triggered_id
return triggered.replace("nav-", "") if triggered else "dashboard"
# --- Callback 3: Open/close modal and score the meal ---
# This handles three different buttons: open, cancel, and score.
# ctx.triggered_id tells us which button was clicked.
@app.callback(
Output("add-meal-modal", "is_open"),
Output("selected-patient-id","data"),
Output("selected-meal-key", "data"),
Input("open-modal-btn", "n_clicks"),
Input("modal-cancel", "n_clicks"),
Input("modal-score", "n_clicks"),
State("modal-patient-select", "value"),
State("modal-meal-select", "value"),
State("add-meal-modal", "is_open"),
State("selected-patient-id", "data"),
State("selected-meal-key", "data"),
prevent_initial_call=True
)
def handle_modal(open_click, cancel_click, score_click,
selected_patient, selected_meal,
is_open, current_patient, current_meal):
triggered = ctx.triggered_id
if triggered == "open-modal-btn":
return True, current_patient, current_meal
if triggered == "modal-cancel":
return False, current_patient, current_meal
if triggered == "modal-score":
if not selected_patient or not selected_meal:
# keep modal open, don't change anything
return True, current_patient, current_meal
# close modal, update patient + meal — render_page will re-fire and score
return False, selected_patient, selected_meal
return is_open, current_patient, current_meal
# --- Callback 3b: Mutually exclusive menu dropdowns ---
# When the user picks a day from one menu, the other three are cleared so only one meal can be selected
at a time.
```

```
@app.callback(
Output("modal-meal-select", "value"),
Output({"type": "menu-dropdown", "period": "March 2026 — Normal"}, "value"),
Output({"type": "menu-dropdown", "period": "March 2026 — Pescetarisk"}, "value"),
Output({"type": "menu-dropdown", "period": "April 2026 — Normal"}, "value"),
Output({"type": "menu-dropdown", "period": "April 2026 — Pescetarisk"}, "value"),
Input({"type": "menu-dropdown", "period": "March 2026 — Normal"}, "value"),
Input({"type": "menu-dropdown", "period": "March 2026 — Pescetarisk"}, "value"),
Input({"type": "menu-dropdown", "period": "April 2026 — Normal"}, "value"),
Input({"type": "menu-dropdown", "period": "April 2026 — Pescetarisk"}, "value"),
prevent_initial_call=True,
)
def sync_meal_select(march_normal, march_pesc, april_normal, april_pesc):
triggered = ctx.triggered_id
if not triggered or not isinstance(triggered, dict):
return None, None, None, None, None

period = triggered.get("period")
values = {
"March 2026 — Normal": march_normal,
"March 2026 — Pescetarisk": march_pesc,
"April 2026 — Normal": april_normal,
"April 2026 — Pescetarisk": april_pesc,
}

selected_value = values.get(period)

# Return the selected value to modal-meal-select,
# keep only the triggered dropdown's value, clear the rest
return (
selected_value,
march_normal if period == "March 2026 — Normal" else None,
march_pesc if period == "March 2026 — Pescetarisk" else None,
april_normal if period == "April 2026 — Normal" else None,
april_pesc if period == "April 2026 — Pescetarisk" else None,
)
# --- Callback 4: Live preview in the modal ---
# This runs whenever the patient or meal dropdown changes in the modal.
# It shows a preview of the grade before the user confirms.
@app.callback(
Output("modal-preview", "children"),
Input("modal-patient-select", "value"),
Input("modal-meal-select", "value"),
)
def update_modal_preview(patient_id, meal_key):
if not patient_id or not meal_key:
return html.P(
"Select a patient and a meal day to see a grade preview.",
style={"color": "#aaa", "fontSize": "13px"
}
)
selected_patient = next((p for p in PATIENTS if p["id"] == patient_id), None)
if not selected_patient:
return None

all_conditions = (
selected_patient["conditions"]["primary"] +
selected_patient["conditions"]["risk"]
)

# Parse the meal key to get pdf_path and day_number
pdf_path, day_number = parse_meal_key(meal_key)

# Try the API first; fall back to local scoring with MEAL_NUTRIENTS if available
api_result = api_score_meal(
pdf_path=pdf_path,
day_number=day_number,
conditions=all_conditions,
patient_id=patient_id,
)
```

```
if api_result:
grade = api_result["grade"]
grade_description = api_result["grade_description"]
source_label = "Live score via API"
source_color = C_GREEN
else:
nutrients = MEAL_NUTRIENTS.get(meal_key, {})
if nutrients:
grade, grade_description = calculate_grade(nutrients, all_conditions)
else:
grade, grade_description = "?", "API offline — no local data for this meal"
source_label = "Local score (API offline)"
source_color = C_ORANGE

grade_color = GRADE_COLORS.get(grade, "#888")

# Build a readable meal label from the key
if "|" in meal_key:
period, day_str = meal_key.rsplit("|", 1)
meal_label = f"{period} · Day {day_str}"
else:
meal_label = meal_key
return html.Div([
html.Hr(style={"borderColor": C_BORDER}),
html.P("PREVIEW", style={
"fontWeight": "bold", "color": C_MUTED,
"fontSize": "10px", "letterSpacing": "1.5px"
}),
html.Div([
html.Div([
html.P(f"Patient: {selected_patient['name']}",
style={"margin": "0", "fontWeight": "bold",
"color": C_TEXT}),
html.P(f"Meal: {meal_label}",
style={"margin": "0", "color": C_MUTED,
"fontSize": "12px"
}),
html.P(f"Conditions: {', '.join(all_conditions) if all_conditions else 'None'}",
style={"margin": "0", "color": C_MUTED, "fontSize": "12px"}),
html.P(source_label, style={
"margin": "4px 0 0 0", "fontSize": "10px",
"color": source_color, "fontWeight": "bold"
})
], style={"flex": "1"}),
html.Div([
html.Span(grade, style={
"fontSize": "48px",
"fontWeight": "bold",
"color": grade_color,
"lineHeight": "1"
}),
html.P(grade_description, style={
"fontSize": "13px", "color": grade_color,
"fontWeight": "bold", "margin": "0"
})
], style={"textAlign": "center"})
], style={
"display": "flex",
"alignItems": "center",
"backgroundColor": "#f7f5f2",
"borderRadius": "8px",
"padding": "12px"
})
])

# --- Callback 5: Patient selection ---
# Handles both the sidebar patient list and the "View dashboard" buttons
# on the Patients page. Both update the selected patient and navigate to
# the dashboard — the only difference is the id prefix of the trigger.
@app.callback(
Output("selected-patient-id", "data", allow_duplicate=True),
```

```
Output("current-page", "data", allow_duplicate=True),
[Input("sidebar-patient-" + p["id"], "n_clicks") for p in PATIENTS] +
[Input("view-patient-" + p["id"], "n_clicks") for p in PATIENTS],
prevent_initial_call=True
)
def select_patient(*args):
triggered = ctx.triggered_id
if triggered:
patient_id = triggered.replace("sidebar-patient-", "").replace("view-patient-", "")
return patient_id, "dashboard"
return PATIENTS[0]["id"], "dashboard"


# ============================================================================
# RUN
# This block only runs when you execute this file directly.
# ============================================================================
if __name__ == "__main__":
import os
debug_mode = os.environ.get("NUTRISCAN_NO_DEBUG") != "1"
print("Starting NutriScan...")
print("Open http://127.0.0.1:8050 in your browser")
app.run(debug=debug_mode)
```

main.py

```
"""
main.py
-------
Entry point for the NutriScan system. Starts both the FastAPI backend and the Dash frontend together
so the whole system runs with one command:

python main.py

Requirements:
- Run from the project root directory
- Use the venv Python: .venv/bin/python main.py

Once running, open http://127.0.0.1:8050 in your browser.
Press Ctrl+C to stop both servers.

------------------------------------------------------------------------
TROUBLESHOOTING
------------------------------------------------------------------------

If ports are already in use, free them first then re-run:

kill -9 $(lsof -ti :8000) 2>/dev/null
kill -9 $(lsof -ti :8050) 2>/dev/null
sleep 2
.venv/bin/python main.py

------------------------------------------------------------------------
RUNNING TESTS
------------------------------------------------------------------------

Run all unit tests from the project root (should see 50 passed):

python -m pytest tests/ -v

------------------------------------------------------------------------
"""

import os
import subprocess
import sys
import time
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent


def _free_port(port: int) -> None:
"""Kill any process already listening on the given port."""
subprocess.call(
f"lsof -ti :{port} | xargs kill -9 2>/dev/null || true",
shell=True
)


def _resolve_python() -> Path:
"""
Return the venv Python executable.
Falls back to sys.executable if the venv sibling binary is not found.
"""
venv_python = Path(sys.executable).parent / "python"
return venv_python if venv_python.exists() else Path(sys.executable)


def main() -> None:
print("=" * 50)
print(" NutriScan — RUC Computer Science Project 2026")
print("=" * 50)
print()
```

```
# Clear ports so we always get a clean start
_free_port(8000)

python = _resolve_python()

# Start the FastAPI backend
print("Starting API server on http://127.0.0.1:8000 ...")
api_process = subprocess.Popen(
[str(python), "-m", "uvicorn", "api.main:app", "--port", "8000"],
cwd=PROJECT_ROOT,
)

# Give the API time to initialise before the dashboard connects
time.sleep(2)

# NUTRISCAN_NO_DEBUG=1 tells dashboard.py to run with debug=False
# so Werkzeug does not try to attach a reloader to a non-interactive process
env = os.environ.copy()
env["NUTRISCAN_NO_DEBUG"] = "1"

# Start the Dash frontend
print("Starting dashboard on http://127.0.0.1:8050 ...")
dashboard_process = subprocess.Popen(
[str(python), "frontend/dashboard.py"],
cwd=PROJECT_ROOT,
env=env,
)

print()
print("Both servers are running.")
print("Open http://127.0.0.1:8050 in your browser.")
print("Press Ctrl+C to stop.")
print()

try:
api_process.wait()
except KeyboardInterrupt:
print("\nShutting down...")
api_process.terminate()
dashboard_process.terminate()
api_process.wait()
dashboard_process.wait()
print("Done.")


if __name__ == "__main__":
main()
```

tests/test_scoring.py

```
import pytest

from pipeline.scoring import score_meal, score_week


# 1. Basic structure test
def test_score_meal_returns_expected_keys():
nutrients = {
"energy_kcal": 650,
"protein_g": 30,
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 500,
"fat_sat_g": 6,
"sugar_g": 8,
}

result = score_meal(nutrients, conditions=[])

assert isinstance(result, dict)
assert "score" in result
assert "grade" in result
assert "flags" in result
assert "nutrient_scores" in result


# 2. Balanced meal should score well (A or B)
def test_balanced_meal_good_grade():
nutrients = {
"energy_kcal": 680,
"protein_g": 25,
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 500,
"fat_sat_g": 6,
"sugar_g": 8,
}

result = score_meal(nutrients, conditions=[])

assert result["grade"] in ["A", "B"]


# 3. High sodium penalty + flag
def test_high_sodium_penalty_and_flag():
nutrients = {
"energy_kcal": 650,
"protein_g": 25,
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 2000, # VERY HIGH
"fat_sat_g": 6,
"sugar_g": 8,
}

result = score_meal(nutrients, conditions=[])

assert result["score"] > 0 # worse score
assert any("Sodium" in flag for flag in result["flags"])


# 4. Protein deficiency → flag
def test_low_protein_flag():
nutrients = {
"energy_kcal": 650,
"protein_g": 5, # VERY LOW
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 500,
```

```
"fat_sat_g": 6,
"sugar_g": 8,
}

result = score_meal(nutrients, conditions=[])

assert any("Protein" in flag for flag in result["flags"])


# 5. Condition multipliers (Hypertension)
def test_hypertension_increases_sodium_penalty():
nutrients = {
"energy_kcal": 650,
"protein_g": 25,
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 1200, # high but moderate
"fat_sat_g": 6,
"sugar_g": 8,
}

result_no_condition = score_meal(nutrients, [])
result_with_condition = score_meal(nutrients, ["Hypertension"])

# Score should be worse with hypertension
assert result_with_condition["score"] > result_no_condition["score"]


# 6. Reward multiplier (Osteoporosis → calcium boost)
def test_osteoporosis_rewards_calcium():
nutrients = {
"energy_kcal": 650,
"protein_g": 25,
"fiber_g": 8,
"calcium_mg": 350, # good calcium
"sodium_mg": 500,
"fat_sat_g": 6,
"sugar_g": 8,
}

result_no_condition = score_meal(nutrients, [])
result_with_condition = score_meal(nutrients, ["Osteoporosis"])

# Lower score is better → should improve
assert result_with_condition["score"] < result_no_condition["score"]


# 7. Multiple conditions stack correctly
def test_multiple_conditions_stack():
nutrients = {
"energy_kcal": 750,
"protein_g": 20,
"fiber_g": 5,
"calcium_mg": 200,
"sodium_mg": 1000,
"fat_sat_g": 10,
"sugar_g": 20,
}

result = score_meal(
nutrients,
["Hypertension", "Type 2 Diabetes"]
)

assert isinstance(result["score"], float)
assert len(result["condition_effects"]) == 2


# 8. Missing nutrients handled as 0
def test_missing_nutrients_defaults_to_zero():
nutrients = {
```

```
"energy_kcal": 650,
# missing others
}

result = score_meal(nutrients, [])

assert isinstance(result["score"], float)


# 9. Grade boundaries test
def test_extremely_unhealthy_meal_gets_bad_grade():
nutrients = {
"energy_kcal": 5000,
"protein_g": 0,
"fiber_g": 0,
"calcium_mg": 0,
"sodium_mg": 10000,
"fat_sat_g": 30,
"sugar_g": 200,
}

result = score_meal(nutrients, [])

assert result["grade"] in ["D", "E"]


# 10. Weekly scoring test
def test_score_week_summary():
meal1 = score_meal({
"energy_kcal": 650,
"protein_g": 25,
"fiber_g": 8,
"calcium_mg": 300,
"sodium_mg": 500,
"fat_sat_g": 6,
"sugar_g": 8,
}, [])

meal2 = score_meal({
"energy_kcal": 800,
"protein_g": 10,
"fiber_g": 3,
"calcium_mg": 100,
"sodium_mg": 1500,
"fat_sat_g": 12,
"sugar_g": 25,
}, [])

weekly = score_week([meal1, meal2])

assert "weekly_grade" in weekly
assert "grade_counts" in weekly
assert weekly["meal_count"] == 2


# 11. Saturated fat penalty cap at 8.0g
def test_saturated_fat_penalty_cap():
nutrients_at_cap = {
"energy_kcal": 650, "protein_g": 25, "fiber_g": 8,
"calcium_mg": 300, "potassium_mg": 1100, "sodium_mg": 500, "fat_sat_g": 8.0, "sugar_g": 8,
}
nutrients_over_cap = {
"energy_kcal": 650, "protein_g": 25, "fiber_g": 8,
"calcium_mg": 300, "potassium_mg": 1100, "sodium_mg": 500, "fat_sat_g": 12.0, "sugar_g": 8,
}

result_at_cap = score_meal(nutrients_at_cap, [])
result_over_cap = score_meal(nutrients_over_cap, [])

# Over the cap should score worse
assert result_over_cap["nutrient_scores"]["fat_sat_g"] > result_at_cap["nutrient_scores"]["fat_sat_g"]
```

```
# Over the cap should raise a saturated fat flag
assert any("Saturated fat" in flag for flag in result_over_cap["flags"])
# At the cap should not raise a flag
assert not any("Saturated fat" in flag for flag in result_at_cap["flags"])


# 12. Grade boundary values
def test_grade_A_boundary():
# Score below -2 should be A
nutrients = {
"energy_kcal": 680, "protein_g": 35, "fiber_g": 10,
"calcium_mg": 400, "sodium_mg": 400, "fat_sat_g": 5, "sugar_g": 6,
}
result = score_meal(nutrients, [])
assert result["grade"] == "A"


def test_grade_E_boundary():
# Score above 13 should be E
nutrients = {
"energy_kcal": 200, "protein_g": 2, "fiber_g": 0,
"calcium_mg": 0, "sodium_mg": 5000, "fat_sat_g": 25, "sugar_g": 100,
}
result = score_meal(nutrients, [])
assert result["grade"] == "E"


def test_grade_C_boundary():
# Adequate meal should land in middle range
nutrients = {
"energy_kcal": 620, "protein_g": 22, "fiber_g": 6,
"calcium_mg": 220, "sodium_mg": 580, "fat_sat_g": 7.5, "sugar_g": 15,
}
result = score_meal(nutrients, [])
assert result["grade"] in ["B", "C", "D"]


# 13. Malnutrition Risk rewards protein and energy
def test_malnutrition_risk_rewards_protein():
nutrients = {
"energy_kcal": 650, "protein_g": 35, "fiber_g": 8,
"calcium_mg": 300, "sodium_mg": 500, "fat_sat_g": 6, "sugar_g": 8,
}
result_no_condition = score_meal(nutrients, [])
result_with_condition = score_meal(nutrients, ["Malnutrition Risk"])

# Malnutrition Risk rewards protein so score should improve
assert result_with_condition["score"] < result_no_condition["score"]


# 14. High Cholesterol penalises saturated fat
def test_high_cholesterol_penalises_sat_fat():
nutrients = {
"energy_kcal": 650, "protein_g": 25, "fiber_g": 8,
"calcium_mg": 300, "potassium_mg": 1100, "sodium_mg": 500, "fat_sat_g": 10.0, "sugar_g": 8,
}
result_no_condition = score_meal(nutrients, [])
result_with_condition = score_meal(nutrients, ["High Cholesterol"])

# High Cholesterol doubles the sat fat penalty
# (Total score may not increase because High Cholesterol also rewards fibre 1.5x,
# which can offset the sat fat penalty. We check the specific nutrient contribution.)
assert result_with_condition["nutrient_scores"]["fat_sat_g"] >
result_no_condition["nutrient_scores"]["fat_sat_g"]
```

tests/test_frida_matcher.py

```
# Import pandas and the functions that will be tested
import pandas as pd

from pipeline.frida_matcher import (
load_frida,
match_ingredient,
match_all_ingredients,
)


# Test exact ingredient matching
def test_match_ingredient_exact_match():

# Sample Frida data
df = pd.DataFrame({
"FødevareNavn": ["oksekød", "kartoffel", "mælk"],
"FoodID": [0, 1, 2],
})

# Run the original function
result = match_ingredient("oksekød", df)

# Check returned values
assert result["query"] == "oksekød"
assert result["matched_name"] == "oksekød"
assert result["frida_id"] == 0
assert result["confidence"] >= 80
assert result["flagged_for_review"] is False


# Test fuzzy matching with a spelling mistake
def test_match_ingredient_fuzzy_match():

df = pd.DataFrame({
"FødevareNavn": ["oksekød", "kartoffel", "mælk"],
"FoodID": [0, 1, 2],
})

result = match_ingredient("kartofel", df)

assert result["matched_name"] == "kartoffel"
assert result["frida_id"] == 1
assert result["flagged_for_review"] is False


# Test matching multiple ingredients
def test_match_all_ingredients_returns_dataframe():

df = pd.DataFrame({
"FødevareNavn": ["oksekød", "kartoffel", "mælk"],
"FoodID": [0, 1, 2],
})

result = match_all_ingredients(
["oksekød", "mælk"],
df
)

assert isinstance(result, pd.DataFrame)
assert len(result) == 2
assert list(result["query"]) == ["oksekød", "mælk"]
assert list(result["matched_name"]) == ["oksekød", "mælk"]


# Test if load_frida cleans invalid food names
def test_load_frida_cleans_food_names(monkeypatch):

# Fake Excel data matching the real Frida structure
def fake_read_excel(*args, **kwargs):
```

```
return pd.DataFrame({
"FødevareNavn": [" oksekød ", "kartoffel", float("nan"), "x"],
"FoodID": [1, 2, 3, 4]
})

monkeypatch.setattr(
pd,
"read_excel",
fake_read_excel
)

result = load_frida("fake_file.xlsx")

# load_frida drops NaN rows but does not strip whitespace from names
# (whitespace stripping happens in nutrient_lookup.py on the Data_Table)
assert "kartoffel" in list(result["FødevareNavn"])
assert len(result) == 3 # " oksekød ", "kartoffel", "x" — NaN row dropped

# Test if low confidence matches are flagged for manual review
def test_low_confidence_flag():

# Create sample Frida data
df = pd.DataFrame({
"FødevareNavn": [
"oksekød",
"kartoffel",
"mælk"
],
"FoodID": [0, 1, 2],
})

# Use a completely different word to force a low confidence score
result = match_ingredient("xxxxxxxx", df)

# Check that the match is flagged for review
assert result["flagged_for_review"] is True


# Test if threshold behaviour works correctly
def test_threshold_behaviour():

# Create sample Frida data
df = pd.DataFrame({
"FødevareNavn": [
"kartoffel"
],
"FoodID": [0],
})

# Small spelling mistake should still give a valid match
result = match_ingredient("kartofel", df)

# Check that confidence is high enough
assert result["confidence"] >= 80

# Check that it is not flagged for review
assert result["flagged_for_review"] is False
```

tests/test_nutrient_lookup.py

```
import pytest
from pathlib import Path

from pipeline.nutrient_lookup import (
load_frida_dataset,
nutrients_for_food,
aggregate_nutrients,
)


# Setup

PROJECT_ROOT = Path(__file__).resolve().parents[2]
FRIDA_PATH = PROJECT_ROOT / "data" / "Frida_5.5_Dataset.xlsx"


# 1. Dataset loads correctly
def test_load_frida_dataset_returns_dataframe():
df = load_frida_dataset()

assert df is not None
assert len(df) > 0


# 2. Known food exists
def test_known_food_in_dataset():
df = load_frida_dataset()

# Pick a known food from Frida (adjust if needed)
assert any("banan" in name.lower() for name in df.index)


# 3. Nutrients return correct structure
def test_nutrients_for_food_structure():
df = load_frida_dataset()

food_name = df.index[0] # safe: first available food
result = nutrients_for_food(food_name, 100)

assert isinstance(result, dict)

expected_keys = [
"energy_kcal",
"protein_g",
"fat_sat_g",
"fiber_g",
"sugar_g",
"sodium_mg",
"potassium_mg",
"calcium_mg",
]

for key in expected_keys:
assert key in result


# 4. Scaling works correctly
def test_nutrients_scaling():
df = load_frida_dataset()

food_name = df.index[0]

nutrients_100g = nutrients_for_food(food_name, 100)
nutrients_50g = nutrients_for_food(food_name, 50)

for key in nutrients_100g:
expected = nutrients_100g[key] / 2
actual = nutrients_50g[key]
```

```
assert abs(expected - actual) < 0.01


# 5. Missing food raises error
def test_nutrients_for_unknown_food():
with pytest.raises(KeyError):
nutrients_for_food("this_food_does_not_exist", 100)


# 6. Aggregation works
def test_aggregate_nutrients():
profiles = [
{"energy_kcal": 100, "protein_g": 10},
{"energy_kcal": 200, "protein_g": 20},
]

result = aggregate_nutrients(profiles)

assert result["energy_kcal"] == 300
assert result["protein_g"] == 30


# 7. Aggregation handles empty list
def test_aggregate_empty():
result = aggregate_nutrients([])

assert result == {}


# 8. Aggregation rounding
def test_aggregate_rounding():
profiles = [
{"energy_kcal": 100.123},
{"energy_kcal": 200.456},
]

result = aggregate_nutrients(profiles)

assert isinstance(result["energy_kcal"], float)
assert round(result["energy_kcal"], 2) == result["energy_kcal"]
```

tests/test_menu_parser.py

```
import pytest

from pipeline.menu_parser import (
clean_sides,
is_vegetarian,
parse_menu,
)


# 1. Sides are cleaned correctly
def test_clean_sides():

result = clean_sides(
"Med kartoffelmos og drueagurker"
)

assert result == [
"kartoffelmos",
"drueagurker"
]


# 2. Vegetarian meals are detected
def test_is_vegetarian_true():

result = is_vegetarian(
"(Vegetarisk) med rodfrugter"
)

assert result is True


# 3. Non-vegetarian meals return False
def test_is_vegetarian_false():

result = is_vegetarian(
"Med kartofler og sovs"
)

assert result is False


# 4. Empty sides return empty list
def test_clean_sides_empty():

result = clean_sides("")

assert result == []


# 5. Connector words are removed
def test_clean_sides_removes_connector_words():

result = clean_sides(
"Med ris og sovs"
)

assert "med" not in result
assert result == ["ris", "sovs"]


# 6. Parse menu returns correct structure
def test_parse_menu_structure(monkeypatch):

fake_rows = [
("MAN Frikadeller", "Æblekage"),
("12 Med kartofler og sovs", ""),
]
```

```
# Fake PDF extraction
def fake_get_rows_from_pdf(pdf_path):
return fake_rows

monkeypatch.setattr(
"pipeline.menu_parser.get_rows_from_pdf",
fake_get_rows_from_pdf
)

result = parse_menu("fake.pdf")

assert isinstance(result, list)
assert len(result) == 1


# 7. Parse menu extracts correct values
def test_parse_menu_values(monkeypatch):

fake_rows = [
("MAN Frikadeller", "Æblekage"),
("12 Med kartofler og sovs", ""),
]

def fake_get_rows_from_pdf(pdf_path):
return fake_rows

monkeypatch.setattr(
"pipeline.menu_parser.get_rows_from_pdf",
fake_get_rows_from_pdf
)

result = parse_menu("fake.pdf")

meal = result[0]

assert meal["day_name"] == "MAN"
assert meal["day_number"] == 12
assert meal["main"] == "Frikadeller"
assert meal["dessert"] == "Æblekage"
assert meal["sides"] == ["kartofler", "sovs"]
assert meal["vegetarian"] is False


# 8. Vegetarian meals are parsed correctly
def test_parse_menu_vegetarian(monkeypatch):

fake_rows = [
("TIR Grøntsagssuppe", "Kage"),
("13 (Vegetarisk) med rodfrugter", ""),
]

def fake_get_rows_from_pdf(pdf_path):
return fake_rows

monkeypatch.setattr(
"pipeline.menu_parser.get_rows_from_pdf",
fake_get_rows_from_pdf
)

result = parse_menu("fake.pdf")

meal = result[0]

assert meal["vegetarian"] is True

# 9. Dessert is extracted correctly
def test_parse_menu_dessert(monkeypatch):
fake_rows = [
("ONS Kylling", "Chokoladekage"),
("14 Med ris og sovs", ""),
]
```

```
def fake_get_rows_from_pdf(pdf_path):
return fake_rows

monkeypatch.setattr(
"pipeline.menu_parser.get_rows_from_pdf",
fake_get_rows_from_pdf
)

result = parse_menu("fake.pdf")

meal = result[0]

assert meal["dessert"] == "Chokoladekage"
```

tests/test_pdf_extractor.py

```
import pytest
from pathlib import Path

from pipeline.pdf_extractor import extract_meals


# Setup
PROJECT_ROOT = Path(__file__).resolve().parents[1]
PDF_PATH = PROJECT_ROOT / "sample_data" / "menuplan-marts-2026.pdf"


# 1. Basic extraction works
def test_extract_meals_returns_list():
meals = extract_meals(str(PDF_PATH))
assert isinstance(meals, list)
assert len(meals) > 0


# 2. Meal structure is correct
def test_meal_has_expected_keys():
meals = extract_meals(str(PDF_PATH))
meal = meals[0]
expected_keys = [
"day_number",
"day_name",
"main",
"sides",
"dessert",
"nutrients",
"matched",
"unmatched",
"coverage_pct",
]
for key in expected_keys:
assert key in meal


# 3. Nutrients structure
def test_nutrients_structure():
meals = extract_meals(str(PDF_PATH))
nutrients = meals[0]["nutrients"]
assert isinstance(nutrients, dict)
for _, value in nutrients.items():
assert isinstance(value, (int, float))


# 4. Coverage percentage valid
def test_coverage_percentage_valid():
meals = extract_meals(str(PDF_PATH))
for meal in meals:
assert 0 <= meal["coverage_pct"] <= 100


# 5. Unmatched ingredients
def test_unmatched_is_list():
meals = extract_meals(str(PDF_PATH))
for meal in meals:
assert isinstance(meal["unmatched"], list)


# 6. Matched ingredients structure
def test_matched_structure():
meals = extract_meals(str(PDF_PATH))
for meal in meals:
for item in meal["matched"]:
assert "ingredient" in item
assert "amount_g" in item
```

```
# 7. File not found (edge case)
def test_extract_meals_file_not_found():
with pytest.raises(Exception):
extract_meals("non_existent_file.pdf")


# 8. Day numbers valid
def test_day_numbers_valid():
meals = extract_meals(str(PDF_PATH))
for meal in meals:
assert isinstance(meal["day_number"], int)
assert meal["day_number"] > 0


# 9. Main dish structure
def test_main_dish_string():
meals = extract_meals(str(PDF_PATH))
for meal in meals:
assert isinstance(meal["main"], str)
assert len(meal["main"]) > 0


# 10. Full pipeline sanity check
def test_full_pipeline_output_reasonable():
meals = extract_meals(str(PDF_PATH))
meal = meals[0]
assert isinstance(meal["nutrients"]["energy_kcal"], (int, float))
assert isinstance(meal["coverage_pct"], int)


# 11. Dessert nutrient contribution
# Desserts in Danish institutional menus are typically cream-based
# (fløde, sukker, æg) and contribute meaningfully to the total energy
# and sugar content of the meal. This is documented in the literature
# on elderly institutional nutrition — desserts are often used
# deliberately to boost energy intake for residents with low appetite.
# This test verifies that the dessert component is actually contributing
# nutrients to the total meal, and is not being silently ignored.
def test_dessert_contributes_nutrients():
meals = extract_meals(str(PDF_PATH))

# Find a meal where the dessert is in dish_ingredients.json
# Day 1 has Citronfromage (citron, æg, sukker, fløde) — known to be in catalogue
day_1 = next((m for m in meals if m["day_number"] == 1), None)
assert day_1 is not None, "Day 1 not found in parsed meals"

# The dessert should have ingredients listed
assert len(day_1["dessert_ingredients"]) > 0, (
"Dessert ingredients are empty — dessert is not contributing to nutrients"
)

# Total meal sugar should be above zero, since Citronfromage contains sukker
# This confirms the dessert's sugar is included in the aggregated total
total_sugar = day_1["nutrients"].get("sugar_g", 0)
assert total_sugar > 0, (
"Sugar is 0 — dessert sugar contribution is missing from meal total"
)

# Total meal energy should reflect both main dish and dessert
# Citronfromage (fløde + sukker + æg) adds meaningful kcal
total_energy = day_1["nutrients"].get("energy_kcal", 0)
assert total_energy > 100, (
"Energy is suspiciously low — dessert energy may not be aggregated"
)
```

requirements.txt

```
pandas
openpyxl
rapidfuzz
pdfplumber
fastapi
uvicorn
dash
dash-bootstrap-components
requests
plotly
pytest
```